Journal of Parallel and Distributed Computing 169 (2022) 301–316

Contents lists available at ScienceDirect

Journal of Parallel and Distributed Computing

www.elsevier.com/locate/jpdc

ZeroCross: A sidechain-based privacy-preserving Cross-chain solution for Monero

Yuxian Li, Jian Weng *, Ming Li, Wei Wu, Jiasi Weng, Jia-Nan Liu, Shun Hu

College of Cyber Security, Jinan University, Guangzhou 510632, PR China

ARTICLE INFO

Article history:
Received 21 February 2022
Received in revised form 4 June 2022
Accepted 15 July 2022
Available online 22 July 2022

Keywords:
Blockchain
Monero
Zero-knowledge
Cryptocurrency exchange

ABSTRACT

Sidechain-based Cross-chain exchange protocols enable payers to exchange cryptocurrencies among different blockchains via a sidechain. Many efforts, such as P2DEX (ACNS' 21), have been proposed to enhance cross-chain exchange privacy protection. However, existing sidechain-based cross-chain solutions for Monero on privacy concerns have limitations: requiring multiple pairs of parties paying simultaneously or fixed transaction amounts. This paper proposes ZeroCross, a novel privacy-preserving sidechain-based scheme that guarantees transaction unlinkability, exchanging fairness, and value confidentiality. ZeroCross designs: (i) a key exchange mechanism that guarantees exchanging fairness and (ii) a verification mechanism that utilizes CP-SNARK to ensure the transaction is confirmed without revealing the details of transactions. In addition, we discuss the influence of the remote side-channel attack in cross-chain exchange and the defence strategy. Finally, we prove the privacy and security of ZeroCross under the Universal Composability (UC) framework and evaluate the practical performance on computation and communication costs.

© 2022 Elsevier Inc. All rights reserved.

1. Introduction

Monero [39], one of the most popular privacy-oriented decentralized cryptocurrencies, protects both identity privacy and transaction value privacy. Recently, the total market value of Monero has reached about 3.96 billion US dollars [29]. Thus, it becomes a natural requirement for Monero to exchange assets with other decentralized cryptocurrencies.

Many cross-chain exchange approaches [27,46,42,36,15,28,34,26,38,1,23] have been widely studied. We elaborate those solutions on a category-by-category basis as follows: (a) *notary schemes* [21,36] rely on single-trusted or multi-trusted to monitor one underlying blockchain and perform corresponding actions on another underlying blockchain after the specified event occurred to complete the cryptocurrency exchange; Corda [21] selects a group of trust parties with well-reputation as notaries to ensure payers conduct cross-chain transactions correctly. Ripple [36] designs a new blockchain system and takes users in this system as notaries to achieve cross-chain exchanges. (b) *Hashed TimeLock Contract* (HTLC) [15,28] requires payers to lock cryptocurrencies in chain A and chain B through smart contracts and allows payers to withdraw cryptocurrencies under certain conditions. PCAS [15] and AMHL [28] focus on protecting the transferred values and the identification of payers based on the HTLC-based protocol. However, this approach is not compatible with the blockchain systems without a smart-contract-like mechanism; (c) *Sidechains/relays* [26,46,41,12,20] are based on a verification mechanism and smart-contract-like mechanism in chain C to verify the events happening in chain A and chain B. Cosmos and Polkadot [26,46] are lack of considering the privacy concerns in Monero. Tari [41] depends on the Tor network [22] which will reduce the communication speed of blockchain users. Manta [12], Xclaim [47], and Haveno [20] require payers to lock assets for a long time to issue tokens in the sidechain.

Except for the above solution, several cross-chain exchanging schemes specifically for Monero have been proposed [23,18,1]. However, those works in practical application are suitable for restricted payment scenes. ASBM and BCAS [23,18] are only applicable to the blockchain supporting smart contracts. P2DEX [1] limits its privacy protection in the situation that several payers initiate a cryptocurrency exchange simultaneously.

A satisfying cross-chain exchange solution should overcome the mentioned limitations to achieve practicability. Compared with (a), (b), and some of the specific solutions for Monero [23,18,1], sidechain-based solutions can be adapted to more scenarios and support more complicated functionalities [38,2]. The main reason

* Corresponding author.
E-mail addresses: yuxianlijnu@gmail.com (Y. Li), cryptjweng@gmail.com (J. Weng), limjnu@gmail.com (M. Li), wuwei2019jnu@gmail.com (W. Wu), wengjiasi@gmail.com (J. Weng), j.n.liu@foxmail.com (J.-N. Liu), gzhushun@gmail.com (S. Hu).

https://doi.org/10.1016/j.jpdc.2022.07.008
0743-7315/© 2022 Elsevier Inc. All rights reserved.

is that they use the underlying protocols (e.g., the consensus protocol) independent of the main chains. This adaptability provides more flexibility for payers to operate their currencies on the main chain according to the functionality provided by the sidechain. Furthermore, the damage (e.g., economic loss and system outage) in the sidechain is limited when it has a cryptographic break or a maliciously designed sidechain. Thus, we are inspired by this widely adopted approach to construct our privacy-preserving cross-chain solution for Monero with the sidechain construction.

The most significant challenge to constructing a sidechain-based solution for Monero is to guarantee the exchange enforcement correctly and address the issues simultaneously, especially considering that a chain's privacy violation would compromise the privacy of another exchanging chain. The sidechain-based solutions enable adversaries to learn the details of transactions from both chains, exposing the transaction's private information, including the linkability of transactions in underlying blockchains and the details of those transactions. That is, the adversaries can utilize the details of the bitcoin transaction to deduce the details of the transaction in Monero, such as obtaining the exchange value of the transaction in Monero based on the exchange value of the transaction in Bitcoin [15]. Additionally, adversaries can launch the remote side-channel attack to eavesdrop on the details of transactions in Monero so that they deduce the details of the related transactions in Bitcoin while payers are executing the cross-chain exchange utilizing the solution mentioned above [44]. Defending against such privacy violations is necessary, but current sidechain-based cross-chain solutions have not resolved it until now.

Our solution. We design a sidechain privacy-preserving cross-chain solution named ZeroCross. It enables the involved payers to proactively prove to an intermediary that they indeed correctly construct the exchanging transactions as agreed in each underlying blockchain without revealing the relationship and details of these transactions. We first design a protocol that enforces payer A to lock Monero in a shared address. If there is a dishonest payer, the other payer will not lose his/her cryptocurrencies because the honest payer can refund his coins through our protocol. Further, we utilize the accumulator and zero-knowledge proof of knowledge to construct a proof stating that a transaction is in a confirmation block without revealing the relationship and details of the exchanging transactions to miners in the sidechain. Specifically, the details of the transactions include the amount and the identification of payers. On the one hand, such proof can show whether payers pay their cryptocurrencies correctly based on this construction. On the other hand, such proof guarantees the miners can not deduce the identification of payers in another underlying blockchain via the identification in Monero captured by launching the remote side-channel attack. In conclusion, based on our solution, payers can correctly proceed in cross-chain exchange and prevent the miners from performing remote side-channel attacks to simultaneously deduce the privacy information from the exchanging transactions. Our solution can be instantiated based on existing blockchains (e.g., Ethereum).

1.1. Contributions

We summarize our contributions in this paper as follows:

- We propose ZeroCross, a privacy-preserving cross-chain cryptocurrency exchange solution for Monero. We utilize the zero-knowledge proof protocol named CP-SNARK to prove a transaction in a confirmation block without revealing the exchange details, such as the receiving addresses, the transaction amount, and the linkability of the transactions. Also, we design a cross-chain protocol to ensure that the honest payer does not lose its coins and obtains the cryptocurrencies when both payers behave honestly.
- We find out the influence of the remote side-channel attack in the cross-chain cryptocurrency exchange. Then, we construct a mechanism based on the zero-knowledge proof of knowledge and the accumulator to defend against the remote side-channel attack.
- We formally prove the security of the ZeroCross in the UC framework and show that it realizes unlinkability, fairness, and confidentiality.
- We design a series of experiments to illustrate the functionality and feasibility of ZeroCross, which show the communication cost, the computation cost, and the latency of our solution.

1.2. Organization

The remaining part of the paper is organized as follows. We present the preliminaries on Monero and some cryptographic primitives in Section 2. In Section 3, we formulate the system architecture. In Section 4, we give a formal security definition of our work. In Section 5, we form the concrete design of our solution. Then, we also discuss the security of our solution in Section 6. In Section 7, we present the experiments and performance evaluation result. In Section 8, we describe the related works about cross-chain payment exchange protocols. Finally, the conclusion and the future work are given in Section 9.

2. Preliminary

This section introduces the necessary preliminaries employed in our design for ease of understanding.

2.1. Monero

Monero is one of the most popular privacy-oriented blockchains [30,45,5]. It protects the private information about transactions, including transaction's senders and receivers and transferred values by the following three cryptographic techniques: *ring signature, stealth addresses, and range proof*. Specifically, ring signatures are used to hide the signer (i.e., sender) of a transaction with a group of public keys so that a verifier cannot identify the actual transaction sender. Stealth addresses, essentially re-randomized public keys, are designed to protect the identity privacy of receivers. Because each stealth address is only used once for each transaction, thus those transactions cannot be linked to any identical receivers. Range proof [7] ensures the correctness of transactions without revealing the transactions' transferred values. Based on this technique, a verifier gets no more information except for being convinced that the transaction's total output value is equal to its total input value.

Here, we further explain Monero blockchain with more detail. Take a user with a private key pair $(a, b) \in \mathbb{Z}_q^2$ as an example; Note that a is the secret key which is used to view all transactions related to the public key pair, and b is the spend key for spending the cryptocurrencies corresponding to (A, B). To receive cryptocurrencies, the user first publishes his/her public key pair $(A, B) = (g^a, g^b)$ to senders. To preserve the user's public key, the senders utilize some stealth addresses instead of the public key pair (A, B) as the receiving address. The stealth address is computed as the formula: $P = g^{\mathcal{H}(A^r)} * B$, where r is a random number and $\mathcal{H}$ is a hash function mapping element to $\mathbb{Z}_q$. To spend the received cryptocurrencies in the stealth address P, the user needs to prove that he/she owns a private key pair (a, b) so that the following formula: $P = g^{\mathcal{H}(g^a * R)+b}$ $(R = g^r)$ holds. Further, to hide the stealth address,

the user selects a stealth address set $\{P_1, \ldots, P, \ldots, P_n\}$ to construct a ring signature over the set so that verifier can not identify which stealth address is used. Also, instead of directly publishing the transferred value, the user constructs a range proof to state that the total value of the output transaction is no more than the total value of the input transaction.

2.2. Cryptographic primitives

Next, we briefly introduce the cryptographic primitives used to build our protocol. We note that $\lambda \in \mathbb{N}$ is denoted as the security parameter.

ElGamal Encryption. The ElGamal Encryption scheme is composed of three algorithms (Gen, Enc, Dec). $(sk, pk) \leftarrow \mathsf{Gen}(1^\lambda)$ is used to generate a pair of keys for encryption and decryption, where λ is denoted by the security parameter. $e \leftarrow \mathsf{Enc}(pk, m)$ is the encryption algorithm for encrypting a plain-text m with a secure key sk. $m \leftarrow \mathsf{Dec}(sk, e)$ is the decryption algorithm to generate a cipher-text e. Note that it is easy to convert the ElGamal encryption scheme as a threshold protocol like [24]. Specifically, the secret key sk is distributed as several shares via a t-out-of-n ($t \leq n/2$) secret sharing protocol [37]. Each miner i holds a share key_{enci}. Decrypting a ciphertext requires at least t miners to participate in the decryption. To simplify our description, we omit more detail on constructing the t-out-of-n ElGamal encryption scheme here.

Digital Signature Scheme. A digital signature scheme is composed by three algorithms (KGen, Sign, Ver). The key generation algorithm $(sk, pk) \leftarrow \mathsf{KGen}(1^\lambda)$ outputs the public key pk and the private key sk on input of the security parameter λ. The signing algorithm $\sigma \leftarrow \mathsf{Sign}(sk, message)$, on input a $messsage$ and a secret key sk, outputs a signature σ. The verification algorithm$\{0, 1\} \leftarrow \mathsf{Ver}(pk, \sigma, message)$ is able to verify the signature. Further, the basic security of a digital signature scheme is EUF-CMA, which means the scheme is existential unforgeability under chosen message attack.

Commitment Scheme. A commitment scheme achieves that a prover can commit a message m to a commitment com without revealing it, convince verifiers that the message m is committed, and open the commitment com to the message m later. We introduce the Pedersen Commitment [33] scheme for committing elements and set commitment scheme [3] for committing a whole set, which includes two algorithms (COM, VCOM). COM is the commitment algorithm such that $(com, decom) \leftarrow \mathsf{COM}(m)$. The verification algorithm $\{0, 1\} \leftarrow \mathsf{VCOM}(com, decom, m)$ takes commitment com, decommitment message $decom$, and message m as inputs, and outputs $\{0, 1\}$ that indicates whether com is a valid commitment on the message m.

Zero-knowledge Proof of Knowledge. Zero-knowledge proof of knowledge (ZKPoK for short) allows a payer $\mathcal{P}$ to prove that he possesses a secure value v which satisfies the relation $\mathcal{R}$ without revealing any additional information of v [32,7]. ZKPoK protocols provide three main properties: 1) *Completeness*: V will accept the proof of any factual statements proven by P; 2) *Soundness*: P can convince V of a false statement in a negligible probability; 3) *Zero Knowledge*: None of the additional information of the secret value will be revealed from the proof π. We introduce the zkSNARK protocol [32] and the Schnoor protocol [35] into our scheme, considering the efficiency and functionality of these protocols. zkSNARK is a non-interactive and universal zero-knowledge proof protocol, which makes it possible to achieve zero-knowledge proof of membership with the accumulator without revealing any information of the secret value. As for the Schnoor protocol, it is an interactive zero-knowledge proof protocol to help us to exchange an element securely.

We notate that $\mathcal{P}$ sends the statement x and a proof π to several verifiers $\mathcal{V}$ for convincing them that v satisfies the relation $\mathcal{R}$ without revealing it. The ZKPoK is denoted as follows:

$$\mathsf{ZKPoK} = \{(v) : \mathsf{Relation}\} \tag{1}$$

The ZKPoK consist of two main algorithms (Proof, VerifyProof). Proof represents the prover algorithm $\pi \leftarrow \mathsf{Proof}(x, w)$. VerifyProof represents the verifier algorithm. $\{0,1\} \leftarrow \mathsf{VerifyProof}(x, \pi)$, where a prover can use π to convince a verifier that there is a witness w for a statement x without revealing any information about w.

Accumulator. It is one of the essential cryptographic primitives to prove that one specific value is included in a particular set [8,14,5,31]. We introduce the accumulator [8] proposed by Camenisch and Lysyanskaya and established on the Strong RSA assumption to prove that a transaction is a member of a block. Following the definition of [8], the Accumulator scheme consists of four algorithms (Setup, Accumulate, GenWitness, Verify): $(N, u) \leftarrow \mathsf{Setup}(\lambda)$, executed by a trusted party, generates a series of public parameters based on the chosen security parameters. This algorithm is only executed one time during the system lifetime. $Acc \leftarrow \mathsf{Accumulate}(\mathcal{X}, (N, u))$, run by payers, computes an accumulator Acc for a set of elements $\mathcal{X} = \{x_1, \ldots, x_n\}$. The payers execute $wit \leftarrow \mathsf{GenWitness}(\mathcal{X}, (N, u), v = x_i)$ to generate a witness wit stating that x_i belongs to $\mathcal{X}$. The verification algorithm $\{0, 1\} \leftarrow \mathsf{Verify}((N, u), wit, Acc, v = x_i)$ takes the accumulator Acc and the witness wit as inputs to certify whether x_i is the member of $\mathcal{X}$ or not.

For the stake of privacy protection, we introduce the accumulator with a non-interactive proof of zero-knowledge protocol CP-SNARK [3] in our solution. CP-SNARK is a zkSNARK-based protocol for proving a committed element included in a certain accumulator without revealing any valid information about the element. Specifically, the non-interactive zero-knowledge proof protocol constructed is as follows:

$$\begin{aligned}\mathsf{ZKPoK} = \{(u_i, U) : &\mathsf{Verify}(wit, Acc, x_i) = 1 \wedge \\ &\mathsf{COM}(u_i) = (com, decom) \wedge \\ &\mathsf{COM}(U) = (com_{set}, decom) \wedge \\ &\mathsf{H}_{\mathsf{prime}}(u_i) = x_i\}.\end{aligned} \tag{2}$$

H_{prime} represents a hash function whose outputs are division intractable. U is represented as a set $\{u_1, \ldots, u_n\}$. Acc is denoted as an accumulator in the set $\{\mathsf{H}_{\mathsf{prime}}(u_1), \ldots, \mathsf{H}_{\mathsf{prime}}(u_n)\}$. The proof indicates that the committed value u_i is included in a set U.

3. System architecture

In this part, we introduce the system model, the threat model, and the design goals of ZeroCross.

3.1. System model

3.1.1. Participants

ZeroCross (cf. Fig. 1) essentially provides a cross-chain cryptocurrency exchange sidechain-based solution specific, aiming to address the privacy concerns for Monero. Specifically, our solution involves the following entities:

- **Payers** possess Monero and other cryptocurrencies like Bitcoin or ZeroCash and wish to exchange their assets via ZeroCross in the network.
- **Miners** are entities capable of maintaining the relay blockchain by verifying and confirming the correctness of the cross-chain payers' operations.

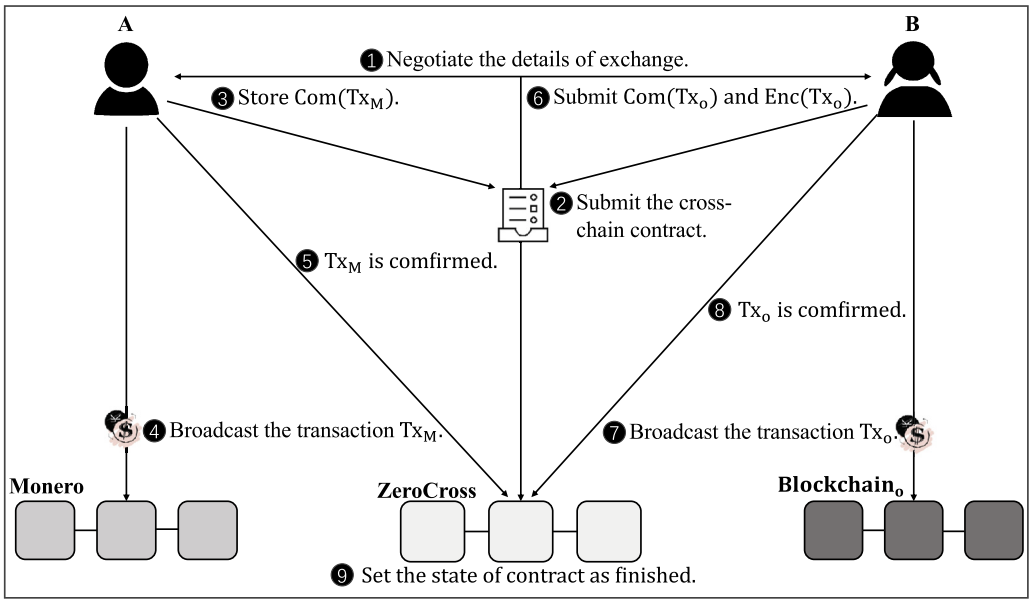


Fig. 1. A high-level overview of ZeroCross architecture.

- **Underlying Blockchains** can be instantiated as Monero, Bitcoin, ZeroCash, Etc. The underlying blockchain is denoted as $\mathsf{Blockchain}_\mathsf{o}$.
- **Relay Blockchain** is a permissioned blockchain, denoted by $\mathsf{Blockchain}_\mathsf{R}$, which issues and transfers cross-chain contracts to record the information of the exchange. The cross-chain contract contains: (**i**) the commitments for the cross-chain transactions in the underlying blockchains; and (**ii**) the proofs stating that the cross-chain transactions have been confirmed on the underlying blockchain.

3.1.2. Interactions

For ease of understanding, we now clarify a cross-chain exchanging process between Monero and $\mathsf{Blockchain}_\mathsf{o}$ via ZeroCross. To begin with, a shared account, controlled by Payer A and Payer B, is created to store certain exchanging Monero from Payer A (①). Both payers then cooperate to initiate a cross-chain contract on the relay blockchain for recording the necessary information to enforce the cross-chain exchange. Payer A can submit the commitment of the transaction to commit that he/she deposits Monero in the shared account. (②-③). After that, Payer A broadcasts the transaction on the Monero. (④). When the transaction is confirmed on the Monero, a proof for the confirmation of this transaction can be submitted to the cross-chain contract on the relay blockchain $\mathsf{Blockchain}_\mathsf{R}$ (⑤). Upon witnessing the proof, Payer B launches a transaction for transferring the related cryptocurrencies in another underlying blockchain to Payer A. Payer B submits the commitment and ciphertext of the transaction into the cross-chain contract (⑥). Similarly, Payer B broadcasts the transaction on $\mathsf{Blockchain}_\mathsf{o}$ (⑦). Next, a proof, stating that the transaction is confirmed on the underlying blockchain is submitted to the cross-chain contract (⑧). Finally, the miners on the $\mathsf{Blockchain}_\mathsf{R}$ authorize Payer B to take complete control of the shared account so that Payer B can obtain the Monero stored before. At that time, the whole exchange process is finished (⑨).

Through ZeroCross, Payer B obtains the Monero *if and only if* Payer A gets the promissory cryptocurrency. If a transaction is failed during the exchanging process, Payer A can redeem his/her Monero, and Payer B would not lose his/her cryptocurrencies.

3.2. Threat model

In this part, we firstly introduce a novel attack named remote side-channel attack in cross-chain exchange. Meanwhile, we describe the trust models of the roles in our scheme.

Remote Side-channel Attack in Cross-chain. This attack originated from a remote side-channel attack [44] against privacy-preserving blockchains. The remote side-channel attack enables an active remote adversary to identify the receiver of any transactions in Monero.

Specifically, since the user acts as the receiver, the user will temporarily trigger the lock mechanism to lock its wallet for further operations. Therefore, if other requests are received at this time, the request's response time will be delayed. Utilizing this phenomenon, a P2P adversary in the network can determine which node is the receiver by sending requests to each node and analyzing its response time. Note that we only consider the case where Monero is vulnerable to the remote side-channel attack.

Next, we extend this attack to the setting where a user exchanges Monero with another cryptocurrency. An attacker (i.e., a corrupted miner in the sidechain) launches this attack against Monero to disclose the receiver's identity in Monero. Since the receiver in Monero is identical to the sender in another underlying blockchain, the linkability of those transactions related to the same cross-chain exchange can enable the adversaries to capture the sender's identity in another blockchain. Additionally, the adversaries can utilize the identity information in the other blockchain to further destroy privacy in Monero. This attack is workable since there is no privacy protection method to guarantee the unlinkability of those transactions related to the same cross-chain exchange. Our work crafts a defence solution against the attack with the idea of breaking the linkability of the transactions issued on the underlying blockchain. Based on the CP-SNARK protocol, users can state the transactions related to a cross-chain exchange are correct and included in confirmation blocks without revealing which confirmed transactions are the actual transactions they used. Thus, the corrupted miner can not capture the relationship of the transactions related to the same cross-chain exchange.

Trust Models. Next, we describe the trust models of the roles as follows:

Table 1
The Notation used in our design.

Notation	Description
$state$	The state of the cross-chain contract.
$addr_{enc}$	The encryption of an address for receiving cryptocurrencies.
(sk_{enc}, pk_{enc})	A pair of keys owned by miners for encryption.
com_m/com_o	The commitment of a transaction.
π_o/π_m	The proof for stating a transaction is included in a confirmation block.
(pk_m, sk_m)	A pair of keys of the common account created by payers.
x/z	The exchanging amount.
$view_{key}$	The key for viewing the balance of the common account created by payers.

- **Payers.** They may steal cryptocurrencies from another payer or fraud miners with incorrect proofs to state that they transfer cryptocurrencies correctly but not.
- **Miners**. We assume that a majority of miners are semi-honest. Miners will honestly follow the defined protocol, but they are also curious about the private information, e.g., the linkability of the transaction issued in the underlying blockchains. They can launch remote side-channel attacks to eavesdrop on that private information. Further, it is emphasized that payers can not collude with most miners.

Remark. We assume that the employed cryptographic primitives, such as cryptographic hashes, accumulators [14,5,31], zero-knowledge protocols [35,32,7] and encryption schemes [16], are secure. Also, we assume that a trusted party is responsible for generating necessary public parameters at the initialization phase.

3.3. Design goals

Our primary goal is to propose a privacy-preserving cross-chain cryptocurrency exchange sidechain-based solution specific to Monero, ensuring the correctness of the exchanging process while protecting the privacy of the exchanging transactions. Based on the above requirements, we summarize three security goals as follows:

- **Fairness.** Payers obtain the exchanging cryptocurrencies at the same time, or none of them does. If all exchanging transactions are processed correctly, both cross-chain payers can receive the cryptocurrencies; otherwise, they can withdraw them without financial loss.
- **Unlinkability.** An exchanging transaction issued on the Monero cannot be linked with a transaction issued on another underlying blockchain against miners on the relay blockchain.
- **Confidentiality.** The miners on the relay blockchain can not obtain the details of the exchanging transactions if the exchange succeeds.

4. Security definition

This section defines the security of the ZeroCross protocol under the UC framework [9]. More specifically, we define the UC model for the communication setting and the global ledger functionality $\mathcal{L}_m$ (and $\mathcal{L}_o$).

4.1. Notation

Before we describe the definition in detail, we start with introducing the notation (cf. Table 1) used here:

- π_m and π_o are generated by payers through executing the proof generation algorithm Proof. Employing these proofs, payers can state that tx_o (or tx_m) is included in a confirmation block.
- (sk_{enc}, pk_{enc}) is a pair of keys for the ElGamal encryption algorithm. Specifically, sk_{enc} is represented as $\{key_{enc1}, ..., key_{encn}\}$ (key_{enci} is owned by miner i) in $\mathsf{Blockchain_R}$.
- $state$ is denoted to describe the life cycle of the cross-chain contract. It can be enumerated as follows:

$$\{\mathsf{START}, \mathsf{AM_f}, \mathsf{BO_f}, \mathsf{FINISHED}\}, \tag{3}$$

"START" means that both payers have already made a preparation for exchanging cryptocurrencies. "$\mathsf{AM_f}$ and $\mathsf{BO_f}$" mean that payers have issued the transactions in the underlying blockchains. Also, "FINISHED" represents that the exchange is completed.

- sk_m is the private key of the shared account in Monero and composed by a pair of scalars $(a, sk_A + sk_B) \in \mathbb{Z}_q^2$. Specifically, $(sk_A + sk_B)$ is the spending private key. Note that $view_{key}$ is a, which can be utilized to view all transactions related to the shared account but can not be used to spend coins.
- pk_m is the public key corresponding to sk_m, and is comprised by $(V, pk_A * pk_B) = (g^a, g^{sk_A} * g^{sk_B})$. Additionally, the stealth address is constructed following the formula:

$$P = g^{H(rV)+sk_A+sk_B} \tag{4}$$

where $\mathcal{H}$: a hash function mapping element to $\mathbb{Z}_q$ and r is a random number.

4.2. UC model

Communication model. We assume a synchronous communication network setting denoted as $\mathcal{F}_{syn}$ (see more details in [9]) so that the protocol under this assumption is executed in discrete rounds. Participants can be aware of the current round, and a message sent current will be delivered to certain parties at the beginning of the next round. The communication network setting allows an adversary to delay a message arbitrarily. It means that an adversary can determine the order of messages arriving in a specific round, but it cannot modify the sending order of messages between honest parties.

The global ledger functionality. The global ledger functionality $\mathcal{L}_m/\mathcal{L}_o$ (cf. Fig. 2) includes publicly available information that can be operated by any other ideal functionalities and protocols. The ledger can store a list of *block* s and each block consists of a set of tuples $(value, destination, source, tx_id)$. In detail, tx_{id} is the identification number of the transaction, $value$ is the amount of the transaction, $source$ is the sender, and $destination$ is the address of the receiver.

4.3. Ideal functionality

The ideal functionality $\mathcal{F}$ in Fig. 3 is regarded as an incorruptible third party to interact with payers $\mathcal{P}_A/\mathcal{P}_B$ and miners $\mathcal{M}$. It formalizes a cross-chain asset exchanging process where $\mathcal{P}_B$ can get x Monero by obtaining the entire private key (sk_A, sk_B) of the

The ideal functionality $\mathcal{L}_m/\mathcal{L}_o$ contains a list $block$ of the tuples tx=($value$, $destination$, $source$, tx_{id}).

Upon receiving (update, cid, tx) from an entity, append (tx) to $block$ and output (updated, cid, $block$).

Upon receiving (read, cid) from an entity, respond with ($block$, cid).

Fig. 2. Ideal functionality $\mathcal{L}_m/\mathcal{L}_o$.

shared account *iff* $\mathcal{P}_B$ sends z coins (*e.g.*, Bitcoin and ZeroCash) to the address $addr_A$. Moreover, the functionality $\mathcal{F}$ can interact with the global ledgers $\mathcal{L}_m$ and $\mathcal{L}_o$ to append transactions into the global ledgers. Here we elaborate on the interaction between honest parties denoted in the ideal functionality $\mathcal{F}$.

Firstly, $\mathcal{P}_A$, $\mathcal{P}_B$, and $\mathcal{M}$ send their inputs to the ideal functionality in round 1. $\mathcal{P}_A$ sends the trading information of the exchanging transactions in Monero (*i.e.,* the transaction tx_m and the transaction amount x) and in $\mathsf{Blockchain_o}$ (*i.e.,* the receiving address $addr_A$ owned by $\mathcal{P}_A$ and the transaction amount z) as inputs to $\mathcal{F}$. Further, $\mathcal{P}_A$ inputs the keys of the exchanging transaction in Monero (*i.e.,* the partial private key sk_A, the corresponding public key $pk_A = g^{sk_A}$, and the view key a) to $\mathcal{F}$. If tx_m does not satisfy x and (g^a, pk_A * pk_B), the ideal functionality will terminate. $\mathcal{P}_B$ inputs the similar information and the differences are sending the partial key sk_B and the transaction tx_o in $\mathsf{Blockchain_o}$ instead of sk_A and tx_m to $\mathcal{F}$. The ideal functionality will terminate while the receiving address of the transaction tx_o is not consistent with $addr_A$. $\mathcal{M}$ also sends the setup message to $\mathcal{F}$.

Next, the functionality $\mathcal{F}$ calls $\mathcal{L}_m$ to append tx_m to the public ledger in the *swap monero* phase. Correspondingly, tx_o is appended to the public ledger by the functionality $\mathcal{F}$ in the *swap ZeroCash* phase. Finally, during the *final determination* phase, there are two possible situations. If tx_m or tx_o is incorrect or not appended to the public ledgers, $\mathcal{P}_A$ will receive sk_B from $\mathcal{F}$. Otherwise, $\mathcal{P}_B$ will receive sk_A from $\mathcal{F}$.

Furthermore, payers can abort the execution of the ideal functionality $\mathcal{F}$ in the *swap monero* phase and *swap ZeroCash* phase. If the corrupted payer $\mathcal{P}_B^*$ aborts the execution in round 3 or round 5, the functionality $\mathcal{F}$ will send sk_B back to $\mathcal{P}_A$. If the execution is aborted by corrupted payer $\mathcal{P}_A^*$ in round 6, $\mathcal{F}$ will continue to instruct the functionality $\mathcal{L}_o$ to append the tx_o to the public ledger. In the end, the $\mathcal{P}_B$ will receive sk_A.

Security Property. Now, we discuss the security properties guaranteed by the ideal functionality $\mathcal{F}$, including fairness, unlinkability, and confidentiality.

Fairness. The payer $\mathcal{P}_A$ obtains z coins from the payer $\mathcal{P}_B$ *if and only if* the payer $\mathcal{P}_B$ receives x coins from the payer $\mathcal{P}_A$.

Unlinkability. If the payers $\mathcal{P}_A$ and $\mathcal{P}_B$ are not corrupted with miners $\mathcal{M}$, the miners $\mathcal{M}$ only ensure that both payers issue correct transaction tx_o and tx_m to $\mathcal{L}_o$ and $\mathcal{L}_m$ but cannot obtain the relationship of tx_o and tx_m.

Confidentiality. The transaction tx_o and tx_m are established by the honest payers so that the miners $\mathcal{M}$ cannot distinguish the transaction tx_o and tx_m from other transaction in the underlying blockchains.

We elaborate on our protocol in the case of a corrupted payer and the case of corrupted miners. Consider first the case of a corrupted payer $\mathcal{P}_A$. The ideal functionality $\mathcal{F}$ proceeds to the *swap monero* phase only if the payers have submitted the correct information during *setup* phase. Then, in the *swap monero* phase, the ideal functionality appends the correct tx_m to the public ledger. Further, we assume instead a corrupted $\mathcal{P}_B^*$, who generates an incorrect transaction tx_o^*. If $tx_o^*.addr_A$ and $tx_m^*.z$ are not consistent with the receiving address $addr_A$ and the amount z, the payer $\mathcal{P}_A$ will obtain the partial secret key sk_B to redeem Monero during the *final determination* phase. In other case (i.e., if tx_o is correct), even if the corrupted payer $\mathcal{P}_A^*$ aborts the execution, the payer $\mathcal{P}_B$ can obtain sk_A during the *final determination* phase. Similarly, if the corrupted payer $\mathcal{P}_B^*$ aborts the execution, sk_B will be sent to $\mathcal{P}_A$ during the last phase, which implies fairness. Finally, we assume the miners $\mathcal{M}$ are corrupted but can not distinguish tx_o and tx_m from other transactions and learn the relationship between the transactions tx_o and tx_m, which implies unlinkability and confidentiality.

The UC-security definition. A protocol Π is admitted to cooperating with the contract ideal functionality $\mathcal{G}$, the global ledger $\mathcal{L}_m$, and the global ledger $\mathcal{L}_o$. We use $\mathsf{exec}_{\Pi,\mathcal{A},\mathcal{E}}$ to denote that the output of an environment $\mathcal{E}$ interacts with adversary $\mathcal{A}$ and parties running the protocol Π. In the ideal world, parties only forward their inputs to an ideal functionality $\mathcal{F}$. The adversary in the ideal world is denoted as a simulator $\mathcal{S}im$, and the output here is denoted as $\mathsf{exec}_{\mathcal{F},\mathcal{S}im,\mathcal{E}}$. Based on the above definition, the security of our protocol can be defined as follows:

Definition 4.1. (UC Security of Π) A protocol Π UC-realize an ideal functionality $\mathcal{F}$ if there exists a ideal adversary $\mathcal{S}im$ for any adversary $\mathcal{A}$ such that any environment $\mathcal{E}$ can not computationally distinguish $\mathsf{exec}_{\mathcal{F},\mathcal{S}im,\mathcal{E}}$ from $\mathsf{exec}_{\Pi,\mathcal{A},\mathcal{E}}$:

$$\mathsf{exec}_{\Pi,\mathcal{A},\mathcal{E}} \approx \mathsf{exec}_{\mathcal{F},\mathcal{S}im,\mathcal{E}}$$

5. Concrete design

Our construction provides a privacy-preserving and verifiable solution for exchanging Monero with other existing cryptocurrencies. Essentially, it addresses two primary concerns at the same time: (a) ensuring the correctness of the exchange; (b) protecting the privacy of the exchanging transactions, including senders, receivers, and transferred values. In light of the two concerns, we first introduce an intermediary blockchain between two exchanging blockchains, a widely adopted method in the existing cross-chain mechanisms. The relay blockchain is used to coordinate and enforce the correct execution of the exchanging transactions. On top of this, we carefully integrate it with a state-of-the-art zero-knowledge proof protocol for generating some proof materials of correct executions while hiding senders, receivers, and the transferred values. Next, we describe the formal description of the protocol.

Before exchanging, the first step is to initialize ZeroCross. Each miner can obtain a partial secret key and public parameters in this part. The miners in $\mathsf{Blockchain_R}$ execute a verifiable secret sharing scheme [10] to create a long-term shared secret key sk_{enc}. After that, each miner i gets a partial key key_{enci}, and the public key pk_{enc} of sk_{enc} is also revealed to payers. Each payer executes $\mathsf{KGen}(1^\lambda)$ to generate a pair of keys for signing signatures and $\mathsf{Gen}(1^\lambda)$ to generate a pair of keys for encryption. Our protocol proceeds in five phases where the protocol participants exchange cryptocurrencies, as shown in Fig. 5. Notice that all following descriptions are based on the assumption that there are two payers

The ideal functionality $\mathcal{F}$ interacts with the parties $\mathcal{P}_A$, $\mathcal{P}_B$, and $\mathcal{M}$, the ideal adversary $\mathcal{S}im$, and the global ledgers $\mathcal{L}_o$ and $\mathcal{L}_m$.

Setup

(Round 1) Upon receiving (setup, cid, z, sk_B, a, $addr_A$, pk_A, x, tx_o) from $\mathcal{P}_B$, if $tx_o.address = addr_A$, store the message and leak (setup, cid, $\mathcal{P}_B$) to Simulator $\mathcal{S}im$.

Upon receiving (setup, cid, x, tx_m, sk_A, $addr_A$, a, pk_B, z) from $\mathcal{P}_A$, if $tx_m.value = x$ and $tx.addr = (g^a, pk_A * pk_B)$ store the message, set $state$ = START and leak (setup, cid, $\mathcal{P}_A$) to Simulator $\mathcal{S}im$; otherwise, terminate.

Upon receiving (setup, cid) from $\mathcal{M}$, store the message and leak (setup, cid, $\mathcal{M}$) to Simulator $\mathcal{S}im$.

Swap Monero

(Round 3) Upon receiving (abort, cid) from the corrupted payer $\mathcal{P}_B^*$, go to the Final Determination.

(Round 4) If no message is received, send (update, cid, $\mathcal{P}_A$, tx_m) to $\mathcal{L}_m$. If $\mathcal{L}_m$ responds with (updated, cid, $block_1$), set $state$ = $\mathsf{AM_f}$, go to Swap ZeroCash phase, and send (updated, cid, $\{tx_i\}_{i \in block_1}$) to $\mathcal{P}_A$.

Swap ZeroCash

(Round 5) Upon receiving (abort, cid) from the corrupted payer $\mathcal{P}_B^*$, go to the Final Determination.

(Round 6) If no message is received or (abort, cid) is received from the corrupted payer $\mathcal{P}_A^*$, send (update, cid, $\mathcal{P}_B$, tx_o) to $\mathcal{L}_o$.
If $\mathcal{L}_o$ responds with (updated, cid, $block_2$), set $state$ = $\mathsf{BO_f}$, go to the Final Determination phase and send (updated, cid, $\{tx_i\}_{i \in block_2}$) to $\mathcal{P}_B$.

Final Determination

(Round 9) If $state$ = $\mathsf{AM_f}$ or $tx_o.value$!= z or $tx_o.receiver$!= $addr_A$, send (final, cid, sk_B) to $\mathcal{P}_A$, else if $state$ = $\mathsf{BO_f}$, send (final, cid, sk_A) to $\mathcal{P}_B$.

Fig. 3. Ideal functionality $\mathcal{F}$.

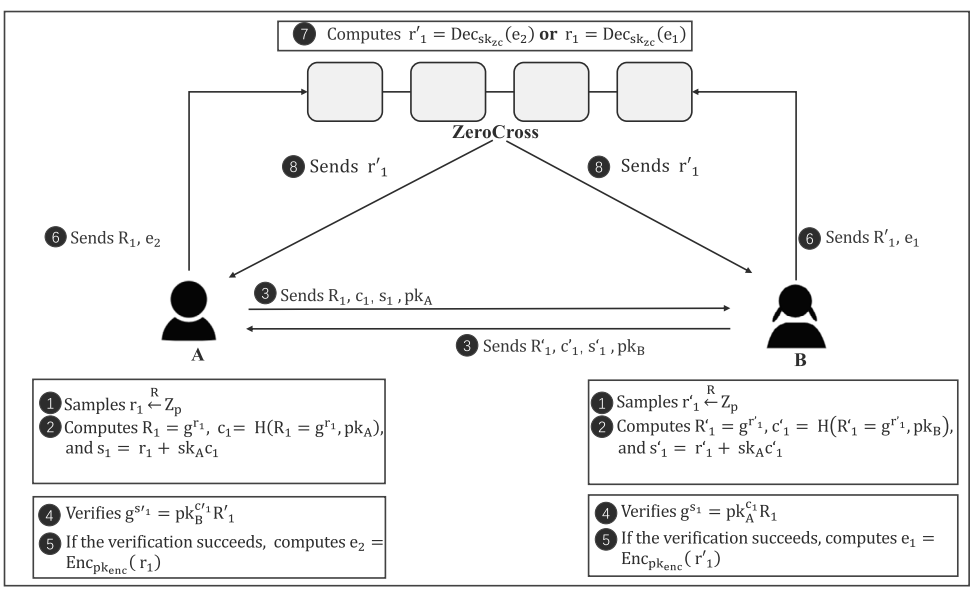


Fig. 4. The process of the key exchange.

($\mathcal{P}_A$ and $\mathcal{P}_B$) establishing a cross-chain contract to swap x XMR owned by $\mathcal{P}_A$ and z ZEC possessed by $\mathcal{P}_B$.

Phase I: Setup. This phase negotiates the details of the exchange and submits the details into the cross-chain contract C (cf. Fig. 6) for enforcing the correctness of the exchange. Doing this naively (i.e., uploading the details of the exchange in plaintext) would leak the private information to adversaries, resulting in using this information to distinguish the exchanging transactions from other transactions. Instead, the private information (e.g., the receiving address $addr_A$ in ZeroCash and the detail of the transaction tx_o) is stored as ciphertext. At the meantime, for guaranteeing the *fairness*, the exchanging Moneros are firstly locked in a common account $address = (g^a, g^{sk_A} * g^{sk_B})$. The private key ($sk_A$, sk_B) of the account is shared by both payers at the beginning, and finally, miners distribute the whole private key to one of the payers based on their execution on the exchange. Specifically, the payer encrypts and sends the partial private key with a correctness proof (c_1, s_i, R_i) to another payer to ensure the payer obtains the correct private

The protocol consists of the behavior of the honest $\mathcal{P}_A$, $\mathcal{P}_B$ and $\mathcal{M}$.

Setup

$\mathcal{P}_A$: **Upon receiving** input (setup, cid, x, tx_m, sk_A, $addr_A$, a, pk_B, z) in round 1, $\mathcal{P}_A$ sends (setup, cid) to $\mathcal{M}$, samples (r, r_1), $R_1 = g^{r_1}$, $c_1 = \mathrm{H}(R_1, pk_A)$, $s_1 = r_1 + sk_A{*}c_1$.
Then, if he receives (cid, pk_{enc}) from $\mathcal{M}$, he computes $e_1 = \mathrm{Enc}\ (pk_{enc}, r_1)$ with pk_{enc}. $\mathcal{P}_A$ sends (setup, s_1, R_1, pk_A, pk_B, cid, e_1, a, x, z, $addr_A$) to $\mathcal{P}_B$, and (setup, cid, e_1, R_1) to $\mathcal{M}$.
Upon receiving(setup, s_1', R_1', cid, pk_A, pk_B, e_2, σ_1, a, x, z, $addr_A$) from $\mathcal{P}_B$, $\mathcal{P}_A$ checks, σ_1, and s_1'. If correct, he sends (stateupdate, START, cid, (e_1, e_2), R_1, R_1') to $\mathcal{G}$ in round 2, otherwise, terminates.
Upon receiving(stop, cid) from $\mathcal{G}$, $\mathcal{P}_A$ terminates.

$\mathcal{P}_B$: **Upon receiving** input (setup, cid, z, tx_o, sk_B, a, $addr_A$, pk_A, x) in round 1, $\mathcal{P}_B$ samples (r_1', r), computes ($addr_{enc}$, z_{enc}) = ($addr_A * r$, $z * r$), $R_1' = g^{r_1'}$, $c_1' = \mathrm{H}(R_1', pk_B)$, $s_1' = r_1' + sk_B{*}c_1'$.
If he receives (cid, pk_{enc}) from $\mathcal{M}$, he computes $e_2 = \mathrm{Enc}(pk_{enc}, r_1')$ and a signature σ_1 on r. He sends (setup, s_1', R_1', pk_A, pk_B, cid, e_2, σ_1, a, x, z, $addr_A$) to $\mathcal{P}_A$, (setup, e_2, cid, R_1') to $\mathcal{M}$, and (init, cid, $addr_{enc}$, z_{enc}) to $\mathcal{G}$.
Upon receiving (stop, cid) from $\mathcal{G}$, $\mathcal{P}_B$ terminates.

$\mathcal{M}$: **Upon receiving** input (setup, cid) in round 1, $\mathcal{M}$ computes (sk_{enc}, pk_{enc}) and sends pk_{enc} to $\mathcal{P}_A$ and $\mathcal{P}_B$.
Upon receiving (setup, cid, e_1, R_1) from $\mathcal{P}_A$ and (setup, e_2, cid, R_1') from $\mathcal{P}_B$, $\mathcal{M}$ decrypts e_1 and e_2 as r_1 and r_1'. If g^{r_1} = R_1 and g^{r_1} = R_1, $\mathcal{M}$ sends (agree, cid), otherwise $\mathcal{M}$ sends (stop, cid).

Swap Monero

$\mathcal{P}_A$: **Upon receiving** (updated, cid) from $\mathcal{G}$ in round 3, $\mathcal{P}_A$ computes (com_m,$decomm$) = COM(tx_m) and sends (updateTxm, cid, com_m) to $\mathcal{G}$.
Upon receiving (updated, cid) from $\mathcal{G}$, he sends (update, tx_m, $\mathcal{P}_A$, cid) to $\mathcal{L}_m$ in round 4.
Upon receiving (updated, $block_1$, cid), $\mathcal{P}_A$ generates (com_{setm}, $decom_{sm}$)=COM($block_1$) and $\pi_m \leftarrow$ Proof(crs, tx_m, com_m, com_{setm}, $block_1$), sends (stateupdate, AM_f, cid, π_m, com_{setm}) to $\mathcal{G}$, and continues to the Swap ZeroCash phase in round 5.

$\mathcal{P}_B$: **Upon receiving** (stop, cid) from $\mathcal{G}$, $\mathcal{P}_B$ terminates.

Swap ZeroCash

$\mathcal{P}_A$: **Upon receiving** (updatedtxo, cid) from $\mathcal{G}$ and (commit, cid, tx_o) from $\mathcal{P}_B$, he proceeds to the Final Determination phase if $tx_o.addr_A = addr_A$ and $tx_o.value = z$, otherwise he sends (dispute, cid, r, σ_1).
Upon receiving (dispute, cid)from $\mathcal{G}$ in round 6, $\mathcal{P}_A$ proceeds to the Final Determination phase.
If (stop, cid) is received from $\mathcal{G}$, $\mathcal{P}_A$ proceeds to the Final Determination phase.
Upon receiving (updatedBlock, cid) from $\mathcal{G}$ in round 7, $\mathcal{P}_A$ sends (read, cid) to $\mathcal{L}_o$ and receives ($block$, cid) from $\mathcal{L}_o$.

$\mathcal{P}_B$: **Upon receiving** (updated, cid) from $\mathcal{G}$ in round 4, $\mathcal{P}_B$ sends (read, cid) to $\mathcal{L}_m$ and receives ($block$, cid) from $\mathcal{L}_m$. Then, if tx_m is not included into $block$, $\mathcal{P}_B$ terminates. Otherwise, $\mathcal{P}_B$ computes (com_o, $decomo$) = COM(tx_o), and sends (updateTxo, com_o, cid) to $\mathcal{G}$ and (commit, cid, tx_o) to $\mathcal{P}_A$.
Upon receiving (disputeTxo, cid), $\mathcal{P}_B$ sends (tx_o, cid, tx_o, $decomo$) to $\mathcal{G}$. **Upon receiving** (continue, cid) from $\mathcal{G}$, $\mathcal{P}_B$ sends (update, cid, tx_o, $\mathcal{P}_B$) to $\mathcal{L}_o$.
Upon receiving (updated, $block_2$, cid, tx_o) from $\mathcal{L}_o$, $\mathcal{P}_B$ generates (com_{seto}, $decom_{so}$)=COM($block_2$), $\pi_o \leftarrow$ Proof(crs, tx_o, com_o, com_{seto}, $block_2$), sends (stateupdate, $\mathsf{BO_t}$, cid, com_{seto}, π_o) to $\mathcal{G}$.

Final Determination

$\mathcal{P}_A$: **Upon receiving** (dispute, cid) from $\mathcal{G}$ in round 6, $\mathcal{P}_A$ sends (finalize, cid, FINISHED) to $\mathcal{G}$.
Upon receiving (r_1', cid) from $\mathcal{G}$ in round 9, he decrypts s_1', outputs (final, sk_B, cid) and terminates.

$\mathcal{P}_B$: **Upon receiving** (updated, cid) from $\mathcal{G}$ in round 7, $\mathcal{P}_A$ sends (finalize, cid, FINISHED) to $\mathcal{G}$.
Upon receiving (r_1, cid) from $\mathcal{G}$ in round 9, he decrypts s_1 as sk_A, outputs (final, sk_A, cid) and terminates.

$\mathcal{M}$: **Upon receiving** (stop, cid) from $\mathcal{G}$ in round 7, $\mathcal{M}$ decrypts e_1' as r_1' and sends (decrypt, cid, FINISHED, r_1') to $\mathcal{G}$.
Upon receiving (decrypt, cid, $\mathcal{P}_A$) from $\mathcal{G}$, $\mathcal{M}$ decrypts e_1' as r_1' and sends (decrypt, cid, FINISHED, r_1') to $\mathcal{G}$.
Upon receiving (decrypt, cid, $\mathcal{P}_B$) from $\mathcal{G}$, $\mathcal{M}$ decrypts e_1 as r_1 and sends (decrypt, cid, FINISHED, r_1) to $\mathcal{G}$.

Fig. 5. Formal protocol description for honest $\mathcal{P}_\mathcal{A}$, $\mathcal{P}_\mathcal{B}$ and $\mathcal{M}$.

key. The encryption keys r_1 and r_2 are submitted to the cross-chain contract C in cipertext (e_1, e_2) so that miners can decrypt them to payers at the last phase.

As shown in Fig. 5, $\mathcal{P}_A$ and $\mathcal{P}_B$ start with negotiating the details of the exchange, including the transferred values x and z, and an address $addr_A$ for receiving ZeroCash. They also exchange a key r for encrypting the receiving address $addr$ and the exchange value z. $\mathcal{P}_A$ and $\mathcal{P}_B$ select the viewing key $view_{key} = a$ together, and select sk_A and sk_B respectively. With partial private keys to the joint account, none of the payers can spend cryptocurrencies in this account. Then, they follow step 1 to step 3 of the key exchange protocol (cf. Fig. 4) to exchange the encryption of their partial keys (sk_A, sk_B). Take $\mathcal{P}_A$ as an example, he/she selects a random number r_1 as the encryption key to encrypt the partial private key sk_A utilizing the formula: $s_1 = r_1 + sk_A * c_1$. $\mathcal{P}_A$ computes and sends the proof (c_1, R_1, s_1) for stating the correctness of the encryption to $\mathcal{P}_B$. The other payer $\mathcal{P}_B$ behaves anomalously. Follow on, they create a cross-chain contract C (cf. Fig. 6) and deploy it into $\mathsf{Blockchain_R}$. In detail, $\mathcal{P}_B$ uploads the encryption z_{enc} and $addr_{enc}$, which will be decrypted in a dispute. The decryption keys for decrypting sk_A and sk_B are encrypted with the following formula:

$$(e_1, e_2) = (Enc(pk_{enc}, r_1), Enc(pk_{enc}, r_1')), \tag{5}$$

where the corresponding private key of pk_{enc} is owned by miners. After that, $\mathcal{P}_A$ uploads the encryption (e_1, e_2), (R_1, R_1') as the

The ideal functionality $\mathcal{G}$ acts as a judge smart contract for session id cid and interacts with the parties $\mathcal{P}_A$, $\mathcal{P}_B$, and $\mathcal{M}$.

Setup

(Round 1) Upon receiving (init, cid, $addr_{enc}$, z_{enc}) from the payer $\mathcal{P}_B$, store $addr_{enc}, z_{enc}$.
(Round 2) Upon Receiving (stateupdate, START, cid, (e_1, e_2), R_1, R_1') from the payer $\mathcal{P}_A$, store (e_1, e_2), set $state$=START. Otherwise, send (stop, cid) to the payers $\mathcal{P}_A$ and $\mathcal{P}_B$, and abort.
Upon receiving (stop, cid) from $\mathcal{M}$, send (stop, cid); otherwise, output (updated, cid).
If no message is received from $\mathcal{P}_A$ or $\mathcal{P}_B$, abort.

Swap Monero

(Round 3) Upon Receiving (updateTxm, cid, com_m) from the payer $\mathcal{P}_A$, store com_m, output (updatedTxm, cid). Otherwise, if no message is received, terminate.
(Round 4) Upon receiving (stateupdate, $\mathsf{AM_f}$, cid, π_m, com_{setm}) from the payer $\mathcal{P}_A$, do the following:

- If VerifyProof(π_m)=1, store π_m and com_{setm}, set $state$=$\mathsf{AM_f}$, and output (updated, cid, π_m).
- Otherwise, send (stop, cid) to the payer $\mathcal{P}_B$, and abort.

Swap ZeroCash

(Round 5) Upon receiving (updateTxo, cid, com_o) from the payer $\mathcal{P}_B$, do the following:

- **Upon receiving** (abort, cid) from the corrupted payer $\mathcal{P}_B$, send (stop, cid) to the payer $\mathcal{P}_A$, set $dispute$ = TRUE and abort.
- Otherwise, store com_o, and send (updatedTxo, cid) to $\mathcal{P}_A$.

(Round 6) Upon receiving (dispute, cid, r, σ_1) from the payer $\mathcal{P}_A$, sends (disputeTxo, cid). Wait until receiving (Txo, cid, tx_o, $decomo$) from $\mathcal{P}_B$, if $tx_o.value$!= z or $tx_o.receiver$!= $addr_A$, set $dispute$=TRUE, and output (dispute, cid); otherwise, continue. If no message is received from the payer $\mathcal{P}_B$, set $dispute$=TRUE, and output (dispute, cid) to $\mathcal{P}_A$.
(Round 7) Upon receiving (stateupdate, $\mathsf{BO_f}$, cid, com_{seto}, π_o) from the payer $\mathcal{P}_B$, do the following:

- If VerifyProof(π_o)=1 and $dispute$=FALSE, store π_o and com_{seto}, set $state$ = $\mathsf{BO_f}$ and output (updatedBlock, cid).
- Otherwise, send (stop, cid) to the payer $\mathcal{P}_A$ and the miners $\mathcal{M}$, and abort.

Final Determination

(Round 8) Upon receiving (finalize, cid, FINISHED) from $\mathcal{P}_B$, if $state$ is equal to $\mathsf{BO_f}$, set $state$=FINISHED, and send (decrypt, cid, $\mathcal{P}_B$) to the miner $\mathcal{M}$.
Upon receiving (finalize, cid, FINISHED) from $\mathcal{P}_A$ when $dispute$ = TURE, set $state$=FINISHED and send (decrypt, cid, $\mathcal{P}_A$) to the miner $\mathcal{M}$.
(Round 9) Upon receiving (decrypt, cid, FINISHED, r_1) from the miners $\mathcal{M}$, store r_1 and output (cid, r_1).
Upon receiving (decrypt, cid, FINISHED, r_1') from the miners $\mathcal{M}$, store r_1' and output (cid, r_1').

Fig. 6. Ideal Functionality $\mathcal{G}$.

proof that the opening of (e_1, e_2) is equal to (r_1, r_1'), and updates the state of the cross-chain contract as START. The miners $\mathcal{M}$ decrypt (e_1, e_2) as (r_1, r_1') to verify (R_1, R_1'). If the verification fails, $\mathcal{M}$ terminates the exchange. It is emphasized that we utilize the committee instead of a single miner to instantiate miners, preventing single-point failure.

Phase II: Swap Monero. $\mathcal{P}_A$ and $\mathcal{P}_B$ begin to exchange cryptocurrencies after the cross-chain contract is deployed. The first step is that $\mathcal{P}_A$ transfers x XMR to the joint account. $\mathcal{P}_A$ constructs tx_m and computes the commitment of tx_m, and then uploads the commitment com_m to the cross-chain contract. $\mathcal{P}_A$ appends tx_m to the global ledger $\mathcal{L}_m$ and constructs a proof π_m for proving that tx_m is packed in a confirmation block.

$$\begin{aligned}\mathsf{ZKPoK} = \{(tx_m, wit) :& \mathsf{Verify}(wit, Acc, e) = 1 \wedge \\ & (com_m, decomm) = \mathsf{Com}(tx_m) \wedge \\ & (com_{setm}, decom_{sm}) = \mathsf{Com}(block_1) \wedge \\ & e = H_{prime}(tx_m)\}\end{aligned} \tag{6}$$

In detail, H_{prime} is a hash function for mapping the element to a prime number, and Verify(wit, Acc, e) is the verification algorithm for verifying whether the element e is included in Acc. Acc is denoted as follows: $Acc = g^{\{H_{prime}(tx_i)\}_{i \in block_1}}$, in which $block_1$ is denoted as the confirmation block consisting of tx_m. At the end, $\mathcal{P}_A$ sends stateupdate with a tuple ("$\mathsf{AM_f}$", π_m) to the cross-chain contract. If $\mathcal{P}_B$ checks if tx_m is correct, they can go on to swap ZeroCash. Otherwise, $\mathcal{P}_B$ will abort the exchange, and $\mathcal{P}_A$ will proceed to the final determination phase.

Phase III: Swap ZeroCash. After swapping Monero, $\mathcal{P}_B$ starts to transfer z ZEC to the address $addr_A$ owned by $\mathcal{P}_A$ in this phase. First of all, $\mathcal{P}_B$ checks the balance of the joint account with $view_{key}$. If the balance is equal to x XMR, $\mathcal{P}_B$ will compute the commitment com_o, upload com_o to the contract and send the transaction tx_o and the decommitment message $decom_o$ to $\mathcal{P}_A$. After that, $\mathcal{P}_A$ verifies whether tx_o is sent to $addr_A$ and the value of it is z. If the verification fails, $\mathcal{P}_A$ will upload the decryption key to the contract for decrypting the $addr_A$ and z and then ask $\mathcal{P}_B$ to open com_o. Next, $\mathcal{P}_B$ responds with the transaction tx_o and the decommitment message $decom_o$. We draw attention to the fact that the amount and address of the transaction in ZeroCash can be opened by opening the commitment in the transaction. If $tx_o.addr_A \neq addr_A$ or $tx_o.z \neq z$, the contract sets $dispute$ = TRUE; otherwise, $\mathcal{P}_B$ sends tx_o to the global ledger $\mathcal{L}_o$. After tx_o is confirmed, $\mathcal{P}_B$ generates a proof for stating tx_o is included in a confirmation block:

$$\begin{aligned}\mathsf{ZKPoK} = \{(tx_o, wit) :& \mathsf{Verify}(wit, Acc, e) = 1 \wedge \\ & (com_o, decomo) = \mathsf{Com}(tx_o) \wedge \\ & (com_{seto}, decom_{so}) = COM(block_2) \wedge \\ & e = H_{prime}(tx_o)\}\end{aligned} \tag{7}$$

where $Acc = g^{\{H_{prime}(tx_i)\}_{i \in block_2}}$ in which $block_2$ is denoted as the confirmation block including tx_o, and Verify is denoted as the verification algorithm verifying tx_o is included in $block_2$.

If no message is sent from $\mathcal{P}_A$, the contract will set the state of the cross-chain contract to "$\mathsf{BO_f}$". That is, $\mathcal{P}_B$ can continue to the next phase if $\mathcal{P}_A$ does not send any disputing information in time. At that time, both payers finish their operation and switch to the last phase.

Phase IV: Final Determination. If both payers finish their operations or make some misbehavior during the exchange, $\mathcal{P}_A$ or $\mathcal{P}_B$ will send a request finalize to finish the last step of the cryptocurrency exchange. In detail, we consider two situations that will happen in the final determination phase: *the normal situation*, which refers to that both payers perform operations as agreed without interruption, and *the unnormal situation*, in which payers are corrupted to make some misbehaviors. There are various responses to different situations.

The normal situation: In this situation, $\mathcal{P}_A$ has stored x XMR in the joint account, and each payer has owned the ciphertext of the one-half private key and another half private key. Meanwhile, $\mathcal{P}_B$ has transferred z ZEC to $\mathcal{P}_B$. Thus, the ownership of the joint account should be delivered to $\mathcal{P}_B$ so that he can obtain x XMR. That is, miners will decrypt the encryption key r_1 with their key_{enci} and upload the r_1 to the contract. $\mathcal{P}_B$ obtains r_1 from the contract and executes the following algorithm Enc (r_1, s_1) for decrypting the partial key sk_A. After that, $\mathcal{P}_B$ owns the whole private key of the joint account and thus can spend cryptocurrencies stored in this account.

The unnormal situation: It can be caused by $\mathcal{P}_B$. The first misbehavior of $\mathcal{P}_B$ is that he/she does not transfer z ZEC in time. In this situation, $\mathcal{P}_A$ can send a request $finalize$ to redeem his/her Monero. The second misbehavior of $\mathcal{P}_B$ is that it constructs an illegal transaction with an incorrect receiver and value. $\mathcal{P}_A$ sends a request *dispute* to redeem his/her Monero while this happens. Specifically, k of n miners utilize their key_{enci} to decrypt the encryption key r'_1 for $\mathcal{P}_A$. $\mathcal{P}_A$ can decrypt the cipher-text of partial key sk_B with r'_1. At that time, $\mathcal{P}_A$ takes ownership of the joint account. It is emphasized that the state of the cross-chain contract will be set as FINISHED after the above operation, referring that the exchange payment is finished and the payment channel between payers is closed.

Remark. We can utilize trusted hardware to build the committee of miners. The miners will perform related operations correctly based on the integrity and confidentiality of trusted hardware.

6. Security analysis

6.1. Main security theorems

The security of ZeroCross is established in Theorem 6.1. Here, the theorem shows that our protocol Π UC-realizes $\mathcal{F}$. Thus, ZeroCross provides fairness, unlinkability, and confidentiality.

Theorem 6.1. *Assuming that the digital signature SIGN is EU-CMA secure, the commitment scheme COM is perfectly binding and computationally hiding, the NIZK scheme is computationally zero-knowledge, simulation sound extractable, and the encryption scheme ENC is perfectly correct and semantically secure. Then, our protocol Π UC-realizes the ideal functionality $\mathcal{F}$ in the $(\mathcal{L}_m, \mathcal{L}_o, \mathcal{G})$-hybrid world.*

Now, we need to prove that the environment $\mathcal{E}$ cannot distinguish the ideal-world execution (with dummy parties $(\hat{\mathcal{P}}_A, \hat{\mathcal{P}}_B, \hat{\mathcal{M}})$ with the ideal adversary $\mathcal{S}im$) from the hybrid-world execution (with $\mathcal{P}_A$, $\mathcal{P}_B$, $\mathcal{M}$ with the adversary $\mathcal{A}$) where each party can interact with the hybrid functionalities $\mathcal{L}_m$, $\mathcal{L}_o$, and $\mathcal{G}$. Specifically, in the ideal world, the dummy parties can only transfer the inputs from the environment $\mathcal{E}$ to the ideal functionality $\mathcal{F}$ and send the outputs from the functionality $\mathcal{F}$ to the environment $\mathcal{E}$. In the real world, the parties execute the protocol Π. The leakage information can be utilized by $\mathcal{E}$. Additionally, the environment $\mathcal{E}$ is capable of selecting the input to parties and obtaining the outputs from parties.

6.2. Construction of the ideal simulator

Next, we demonstrate the construction of the simulator $\mathcal{S}im$ by illustrating what actions $\mathcal{S}im$ should make after observing instructions from $\mathcal{E}$. That is, the simulator $\mathcal{S}im$ should take some actions to prevent the environment $\mathcal{E}$ from distinguishing the execution. We will discuss the construction of $\mathcal{S}im$ in five cases: 1) the execution with honest parties, 2) the execution with the corrupted payer $\mathcal{P}_A$, 3) the execution with the corrupted payer $\mathcal{P}_B$, 4) the corrupted miners $\mathcal{M}$, and 5) three corrupted parties.

CASE I: *The Execution with Honest Parties.* In this case, the dummy parties $\hat{\mathcal{P}}_A$, $\hat{\mathcal{P}}_B$, and $\hat{\mathcal{M}}$ transfer all messages from the environment $\mathcal{E}$ to the ideal functionality $\mathcal{F}$ in the ideal world.

Claim 6.2. *It exists an algorithm $\mathcal{S}im^{hon}$ such that the environment $\mathcal{E}$ cannot computationally distinguish the execution of the protocol Π with the adversary $\mathcal{A}$ in the $(\mathcal{L}_m, \mathcal{L}_o, \mathcal{G})$-hybrid world from the execution of $\mathcal{F}$ with the ideal adversary $\mathcal{S}im$ that **does not corrupt any party**.*

Proof. In this case, $\mathcal{S}im^{hon}$ is only required to generate a transcript of all messages of the execution of Π towards $\mathcal{A}$ and $\mathcal{E}$. The transcript includes the simulation of the protocol message among $\mathcal{P}_A$, $\mathcal{P}_B$, and $\mathcal{M}$, and all interactions with $(\mathcal{L}_m, \mathcal{L}_o, \mathcal{G})$. In this case, we assume there is a secure channel among $\mathcal{P}_A$, $\mathcal{P}_B$, and $\mathcal{M}$. In the secure channel, the environment only knows that messages are being sent but the environment does not know the content of those messages. We define a simulator $\mathcal{S}im^{hon}$ as follows:

1. If $\mathcal{P}_B$ starts the execution with $\mathcal{F}$ in round 1, $\mathcal{S}im^{hon}$ learns (setup, cid, $\mathcal{P}_B$) from $\mathcal{F}$. $\mathcal{S}im^{hon}$ selects r^*, z^*, and $addr^A*$, computes $(addr^*_{enc}, z^*_{enc}) = (addr^* * r^*, z^* * r^*)$, and sends (init, $addr^*_{enc}$, z^*_{enc}) to $\mathcal{G}$.
2. If $\mathcal{P}_A$ starts the execution with $\mathcal{F}$ in round 1, $\mathcal{S}im^{hon}$ learns (setup, cid, $\mathcal{P}_A$) from $\mathcal{F}$. Then, $\mathcal{S}im^{hon}$ samples $r^*_1 \leftarrow Z_p$, generates $(sk^*_1, pk^*_1) \leftarrow \mathsf{Gen}(1^\lambda)$, selects sk^*_A, g^*, h^*, and computes $e^*_1 = \mathsf{Enc}(pk^*, r^*_1)$, $pk^*_A = g^{sk^*_A}$, $R^*_1 = g^{r^*_1}$, $c^*_1 = \mathsf{H}(R^*_1, pk^*_A)$, $s^*_1 = r^*_1 + sk^*_A * c^*_1$. Next, $\mathcal{S}im^{hon}$ continues to sample $r'^*_1 \leftarrow Z_p$, selects sk^*_B, computes $e^*_2 = \mathsf{Enc}(pk^*, r'^*_1)$, $pk^*_B = g^{sk^*_B}$, $R'^*_1 = g^{r'^*_1}$, $c'^*_1 = \mathsf{H}(R'^*_1, pk^*_B)$, $s'^*_1 = r'^*_1 + sk^*_B * c'^*_1$, generates a signature on r^*, and sends (stateupdate, START, cid, (e^*_1, e^*_2), R^*_1, R'^*_1) to $\mathcal{G}$.
3. Upon learning (setup, cid, $\mathcal{M}$) from $\mathcal{F}$, $\mathcal{S}im^{hon}$ sends (agree, cid) to $\mathcal{G}$ in round 2.
4. In round 3, $\mathcal{S}im^{hon}$ constructs tx^*_m based on (sk^*_A, sk^*_B), computes the commitment com^*_m of tx^*_m, and sends (updateTxm, cid, com^*_m) to $\mathcal{G}$.
5. In round 4, $\mathcal{S}im^{hon}$ computes the commitment com^*_{setm} of $\{tx_i\}_{i \in block_1}$, constructs $\pi^*_m \leftarrow \mathsf{Proof}(crs, tx^*_m, com^*_m, com^*_{setm}, \{tx_i\}_{i \in block_1})$. It sends (stateupdate, AM_f, cid, com^*_{setm}, π^*_m) to $\mathcal{G}$.
6. In round 5, $\mathcal{S}im^{hon}$ constructs tx^*_o based on $addr^*$ and z^*, computes the commitment com^*_o of tx^*_o, and sends (updateTxo, cid, com^*_o).
7. In round 6, $\mathcal{P}_A$ and $\mathcal{P}_b$ behave honestly. Thus, no message needs to be constructed, and $\mathcal{S}im^{hon}$ proceeds to round 7.
8. In round 7, $\mathcal{S}im^{hon}$ computes com^*_{seto} of $\{tx_i\}_{i \in block_2}$, constructs $\pi^*_o \leftarrow \mathsf{Proof}(crs, tx^*_o, com^*_o, com^*_{seto}, \{tx_i\}_{i \in block_2})$. Finally, it sends (stateupdate, BO_f, cid, com^*_{seto}, π^*_o) to $\mathcal{G}$.

9. In round 8, $\mathcal{S}im^{hon}$ sends (finalize, *cid*, FINISHED) to $\mathcal{G}$.
10. In round 9, $\mathcal{S}im^{hon}$ sends (decrypt, *cid*, FINISHED, $r_1'^*$) to $\mathcal{G}$.

Running $\mathcal{S}im^{hon}$ in the $\mathcal{F}$ ideal world makes a series of actions to guarantee that the environment $\mathcal{E}$ can not distinguish the $(\mathcal{L}_m, \mathcal{L}_o, \mathcal{G})$-hybrid world execution from the ideal world execution unless $\mathcal{E}$ can decrypt $(addr^*_{enc}, z^*_{enc})$ as $(addr^*, z^*)$ where $addr^* \neq addr_A$ and $z^* \neq z$, and (com^*_m, com^*_o) as (tx^*_m, tx^*_o) where $tx^*_m \neq tx_m$ and $tx^*_o \neq tx_o$. We can show that this only happens if $\mathcal{E}$ breaks the hiding property of the commitment and encryption scheme, which is computationally hard. □

CASE II: *The Execution with the Corrupted Payer $\mathcal{P}^*_A$.* The simulator not only simulates the transcript of the protocol Π but also simulates all outputs of the corrupted payer $\mathcal{P}^*_A$ towards $\mathcal{F}$ and $\mathcal{E}$. That is, whenever the environment $\mathcal{E}$ sends a message through the corrupted dummy party $\mathcal{P}^*_A$, this message is also sent to $\mathcal{S}im^{\mathcal{P}_A}$. Further, based on these inputs, $\mathcal{S}im^{\mathcal{P}_A}$ simulates the execution of Π while interacting with $\mathcal{F}$ in the name of $\mathcal{P}^*_A$.

Claim 6.3. *It exists an algorithm $\mathcal{S}im^{\mathcal{P}_A}$ such that the environment $\mathcal{E}$ cannot computationally distinguish the execution of the protocol Π with an adversary $\mathcal{A}$ in the $(\mathcal{G}, \mathcal{L}_m, \mathcal{L}_o)$-hybrid world from the execution of $\mathcal{F}$ with an ideal adversary $\mathcal{S}im^{\mathcal{P}_A}$. Here, the adversary* ***only corrupts the payer*** *$\mathcal{P}^*_A$.*

Proof. We define a simulator $\mathcal{S}im^{\mathcal{P}_A}$ imitating the execution of Π in the real world with the inputs of the corrupted payer $\mathcal{P}^*_A$. The details of the simulator are as follows:

1. Upon receiving (setup, *cid*, $\mathcal{P}_B$) in round 1, $\mathcal{S}im^{\mathcal{P}_A}$ generates $(key^*_{enc}, pk^*_{enc})$ and sends (cid, pk^*_{enc}) to $\mathcal{P}^*_A$. Also, $\mathcal{S}im^{\mathcal{P}_A}$ receives (setup, s_1, R_1, pk_A, pk_B, *cid*, e_1, *a*, *x*, *z*, $addr_A$) over $\mathcal{P}^*_A$ in round 1. After that, it samples $r_1'^*$, $r^* \leftarrow Z_p$, extracts sk_B from pk_B, and e^*_2 = Enc(pk^*_{enc}, $r_1'^*$), $R_1'^* = g^{r_1'^*}$, $c_1'^*$ = H($R_1'^*$, pk_B), $s_1'^* = r_1'^*$ + sk_B * $c_1'^*$, and generates a signature σ^*_1 on r^*. $\mathcal{S}im^{\mathcal{P}_A}$ sends (setup, s_1', $R_1'^*$, pk_A, pk_B, *cid*, e^*_2, σ_1, $addr_A$) to $\mathcal{P}^*_A$. Then, it simulates the execution of the hybrid functionality by sending (updated, *cid*) = $\mathcal{G}$ (stateupdate, START, *cid*, (e_1, e^*_2), R_1, $R_1'^*$). $\mathcal{S}im^{\mathcal{P}_A}$ extracts sk_A from s_1 and constructs tx^*_m based on pk_A, pk_B, *a* and *x*. At last, it sends (setup, *cid*, *x*, *z*, tx^*_m, sk_A, $addr_A$, *a*, pk_B) to $\mathcal{F}$.
2. Upon receiving (updateTxm, *cid*, com_m) from $\mathcal{P}^*_A$, $\mathcal{S}im^{\mathcal{P}_A}$ runs $\mathcal{G}$ on the inputs (updateTxm, *cid*, com_m) and sends the output of it to $\mathcal{P}^*_A$.
3. Upon receiving (stateupdate, $\mathcal{AM}_f$, *cid*, π_m, com_{setm}) from $\mathcal{P}^*_A$, $\mathcal{S}im^{\mathcal{P}_A}$ runs $\mathcal{G}$ on the inputs (stateupdate, $\mathcal{AM}_f$, *cid*, π_m, com_{setm}) and sends the output of it to $\mathcal{P}^*_A$.
4. In round 5, $\mathcal{S}im^{\mathcal{P}_A}$ constructs tx^*_o based on *z* and $addr_A$, computes $(com_o, decom)$ = COM(tx^*_o), and sends (commit, *cid*, tx_o) to $\mathcal{P}^*_A$.
5. Upon receiving (dispute, *cid*, *r*, σ_1) over $\mathcal{P}^*_A$, $\mathcal{S}im^{\mathcal{P}_A}$ runs $\mathcal{G}$ on the inputs (dispute, *cid*, *r*, σ_1) and sends the output of it to $\mathcal{P}^*_A$.
6. Upon receiving (final, *cid*, FINISHED) over $\mathcal{P}^*_A$, $\mathcal{S}im^{\mathcal{P}_A}$ triggers $\mathcal{F}$ to output (final, *cid*, sk_A) to $\mathcal{P}_B$.

Since $\mathcal{S}im^{\mathcal{P}_A}$ is required to guarantee that $\mathcal{F}$ can output (final, *cid*, sk_A) to the honest $\mathcal{P}_B$, $\mathcal{S}im^{\mathcal{P}_A}$ requires a extractor to extract the sk_A from s_1 and sk_B from pk_B in step (1). Thus, $\mathcal{S}im^{\mathcal{P}_A}$ can simulate the execution of the adversary so that the environment $\mathcal{E}$ can not distinguish the $(\mathcal{L}_m, \mathcal{L}_o, \mathcal{G})$-hybrid world execution from the ideal world execution. □

CASE III: *The Execution with the Corrupted Payer $\mathcal{P}^*_B$.* We will discuss the simulation with the corrupted payer $\mathcal{P}^*_B$, in which the simulator simulates not only the transcript of the protocol Π but also all outputs of the corrupted payer $\mathcal{P}^*_B$ towards $\mathcal{F}$ and $\mathcal{E}$. Whenever the environment $\mathcal{E}$ sends a message through the corrupted dummy party $\mathcal{P}^*_B$, this message is transferred to $\mathcal{S}im^{\mathcal{P}_B}$. Based on these inputs, $\mathcal{S}im^{\mathcal{P}_B}$ simulates the execution of Π while interacting with $\mathcal{F}$ in the name of $\mathcal{P}^*_B$.

Claim 6.4. *It exists an algorithm $\mathcal{S}im^{\mathcal{P}_B}$ such that the environment $\mathcal{E}$ cannot computationally distinguish the execution of the protocol Π with the adversary $\mathcal{A}$ in the $(\mathcal{G}, \mathcal{L}_m, \mathcal{L}_o)$-hybrid world from the execution of $\mathcal{F}$ with an ideal adversary $\mathcal{S}im^{\mathcal{P}_B}$. Here, the adversary* ***only corrupts the payer*** *$\mathcal{P}^*_B$.*

Proof. The simulator in this case needs to simulate the execution of Π in the real world with the inputs of the corrupted payer $\mathcal{P}^*_B$. We define a simulator $\mathcal{S}im^{\mathcal{P}_B}$ as follows:

1. Upon receiving (setup, *cid*, $\mathcal{P}_A$), $\mathcal{S}im^{\mathcal{P}_B}$ generates $(key^*_{enc}, pk^*_{enc})$ and sends (cid, pk^*_{enc}) to $\mathcal{P}^*_A$. Then, if $\mathcal{S}im^{\mathcal{P}_B}$ receives (setup, s_1', R_1', pk_A, pk_B, *cid*, e_2, σ_1, *a*, *x*, *z*) over $\mathcal{P}^*_B$, it samples $r^*_1 \leftarrow Z_p$, extract sk_B from pk_B and sk^*_A from pk_A, and computes e^*_1 = Enc(pk^*_1, r^*_1), $R^*_1 = g^{r^*_1}$, c^*_1 = H(R^*_1, pk_A), and $s^*_1 = r^*_1$ + sk_A * c^*_1. $\mathcal{S}im^{\mathcal{P}_B}$ sends (setup, s_1, R^*_1, pk_A, pk_B, *cid*, e^*_1, $addr_A$) to $\mathcal{P}^*_B$. Upon receiving (init, *cid*, $addr_{enc}$, z_{enc}) over $\mathcal{P}^*_B$, $\mathcal{S}im^{\mathcal{P}_B}$ runs $\mathcal{G}$ on the inputs (init, *cid*, $addr_{enc}$, z_{enc}) and sends the outputs of it to $\mathcal{P}^*_B$. $\mathcal{S}im^{\mathcal{P}_B}$ constructs tx^*_o based on $addr_A$ and *z*, and sends (setup, *cid*, *x*, *z*, tx^*_o, sk_B, $addr_A$, *a*, pk_A) to $\mathcal{F}$ in the name of $\mathcal{P}^*_B$.
2. Upon receiving (updateTxo, com_o, *cid*) over $\mathcal{P}^*_B$, $\mathcal{S}im^{\mathcal{P}_B}$ runs $\mathcal{G}$ on the inputs (updateTxo, com_o, *cid*) and sends the outputs of it to $\mathcal{P}^*_B$.
3. Upon receiving (Txo, *cid*, tx_o, *decomo*) over $\mathcal{P}^*_B$, $\mathcal{S}im^{\mathcal{P}_B}$ runs $\mathcal{G}$ on the inputs (Txo, *cid*, tx_o, *decomo*) and sends the outputs of it to $\mathcal{P}^*_B$.
4. Upon receiving (stateupdate, $\mathcal{BO}_f$, π_o, com_{seto}. *cid*) over $\mathcal{P}^*_B$, $\mathcal{S}im^{\mathcal{P}_B}$ runs $\mathcal{G}$ on the inputs (stateupdate, $\mathcal{BO}_f$, π_o, com_{seto}, *cid*) and sends the outputs of it to $\mathcal{P}^*_B$.
5. Wait until round 9, upon receiving (abort, *cid*) over $\mathcal{P}^*_B$, $\mathcal{S}im^{\mathcal{P}_B}$ triggers $\mathcal{F}$ to output (final, *cid*, sk_B) to $\mathcal{P}_A$.
6. Upon receiving (final, *cid*, FINISHED) over $\mathcal{P}^*_B$, $\mathcal{S}im^{\mathcal{P}_A}$ triggers $\mathcal{F}$ to output (final, *cid*, sk_A) to $\mathcal{P}_B$.

Since $\mathcal{S}im^{\mathcal{P}_B}$ is required to guarantee that $\mathcal{F}$ outputs (final, *cid*, sk_B) to the honest $\mathcal{P}_A$ in step (4) and (final, *cid*, sk_A) to $\mathcal{P}_B$ in step (5), $\mathcal{S}im^{\mathcal{P}_A}$ needs an extractor to extract the sk_A and sk_B in step (1). Based on these settings, the execution of Π in the $(\mathcal{L}_m, \mathcal{L}_o, \mathcal{G})$-hybrid world is indistinguishable from the execution in $\mathcal{F}$ ideal world. □

CASE IV: *The Execution with the Corrupted Miners $\mathcal{M}^*$.* Simulation with the corrupted miners $\mathcal{M}^*$ is a case in which the parties $\mathcal{P}_A$ and $\mathcal{P}_B$ will transfer all messages from the environment $\mathcal{E}$ in an ideal world. All outputs of corrupted miners $\mathcal{M}^*$ also will be required to be simulated. Based on these inputs, $\mathcal{S}im^{\mathcal{M}}$ simulates the execution of Π while interacting with $\mathcal{F}$ in the name of $\mathcal{M}^*$.

Claim 6.5. *It exists an algorithm $\mathcal{S}im^{\mathcal{M}}$ such that the environment $\mathcal{E}$ cannot computationally distinguish the execution of the protocol Π with an adversary $\mathcal{A}$ in the $(\mathcal{G}, \mathcal{L}_m, \mathcal{L}_o)$-hybrid world from the execution of $\mathcal{F}$ with an ideal adversary $\mathcal{S}im$ who* ***only corrupts the miner*** *$\mathcal{M}^*$.*

Proof. The simulator, in this case, needs to simulate the execution of Π in the real world with the inputs of the corrupted payer $\mathcal{M}^*$. We define a simulator $\mathcal{S}im^{\mathcal{M}}$ as follows:

1. Upon receiving (pk_{enc}) from $\mathcal{M}^*$, $\mathcal{S}im^{\mathcal{M}}$ samples r_1^* and $r_1'^*$, computes the encryption (e_1^*, e_2^*) of r_1^* and $r_1'^*$ with (pk_{enc}), and sends (setup, cid, e_1, R_1) and (setup, cid, e_2, R_1') to $\mathcal{M}^*$. Then, $\mathcal{S}im^{\mathcal{M}}$ runs $\mathcal{G}$ on the input (agree, cid) and sends the outputs of it to $\mathcal{M}^*$ while receiving (agree, cid) from $\mathcal{M}^*$. $\mathcal{S}im^{\mathcal{M}}$ sends (setup, cid) to $\mathcal{F}$ in the name of $\mathcal{P}_B^*$.
2. $\mathcal{S}im^{\mathcal{M}}$ runs $\mathcal{G}$ and sends the outputs of it to $\mathcal{M}^*$ when receiving (decrypt, cid, FINISHED, r_1).
3. Upon receiving (decrypt, cid, FINISHED, r_1'), $\mathcal{S}im^{\mathcal{M}}$ runs $\mathcal{G}$ and sends the outputs of it to $\mathcal{M}^*$. □

Since running the $\mathcal{S}im^{\mathcal{M}}$ in the hybrid world guarantees that (r_1, r_1') in the hybrid world is identical with (r_1, r_1') in the real world, the execution of the protocol Π with a adversary $\mathcal{A}$ in the $(\mathcal{G}, \mathcal{L}_m, \mathcal{L}_o)$-hybrid world is indistinguishable from the execution of $\mathcal{F}$ with an ideal adversary $\mathcal{S}im^{\mathcal{M}}$.

CASE V: *Three corrupted parties.* The case of three corrupted parties does not guarantee security, including fairness, unlinkability, and confidentiality. However, we still need to demonstrate that the environment can not distinguish the ideal world and the hybrid world, which guarantees security when our protocol is composed of other protocols.

Claim 6.6. *It exists an algorithm $\mathcal{S}im^{all}$ such that the environment $\mathcal{E}$ cannot computationally distinguish the execution of the protocol Π with an adversary $\mathcal{A}$ in the $(\mathcal{G}, \mathcal{L}_m, \mathcal{L}_o)$-hybrid world from the execution of $\mathcal{F}$ with an ideal adversary $\mathcal{S}im^{\mathcal{P}_A}$ that* ***corrupts the payers*** $\mathcal{P}_A^*$ ***and*** $\mathcal{P}_B^*$ ***and the miner*** $\mathcal{M}^*$.

Proof. The simulation, in this case, is a mixture of the three previous simulations. The simulator is required to simulate the execution of Π in the real world with the inputs of $\mathcal{P}_A^*$, $\mathcal{P}_B^*$, and $\mathcal{M}^*$. Since this case repeats the steps of the simulations with one malicious party, we omit the specific simulation of it here. □

7. Performance evaluation

In this section, we evaluate the performance of our protocol ZeroCross through multiple experiments in terms of computation cost, communication cost, and time consumption. All of our experiments are established on a Laptop with a 3.60 GHz AMD Ryzen 5 3600 6-Core Processor CPU and 8 GB of RAM. We use the CP-SNARK Library [19] to construct the proof of membership protocol [3], and the Java Pairing-Based Cryptography Library (JPBC) to implement the signature scheme and the encryption scheme. We take ZeroCash and Monero as the underlying blockchains to simulate the cross-chain exchange among payers and miners. Concretely, we analyze the influence of the operations, including generating and verifying zero-knowledge proofs, computing, and transferring various parameters. Further, we also discuss the choice of the underlying blockchains.

7.1. Computation cost

Time consumption for each phase. We evaluate the main computation cost for $payer_A$, $payer_B$, and *miners* in ZeroCross (as shown in Table 2) with regard to several steps of ZeroCross and further compare ZeroCross with Xclaim [47]. It is emphasized that all experiments are performed 200 times in a testing environment. Specifically, we evaluate the computation time of each phase to analyze the computation cost in each phase. Based on the current throughput of cryptocurrency, we set the number of transactions in each block of each underlying blockchain as 200 and the proportion of transactions related to the cross-chain exchange payment in each block as 50%. As shown in Table 2, generating a proof of membership is the main computation cost of $payer_A$ in the Swap Monero phase. The main computation cost of *miners* is to verify the proof of membership from $payer_A$. As for miners, the main computation cost is caused by verifying the proof and signature which is generated by payers in the Swap Monero phase and Swap ZeroCash. Finally, compared with the total computation time of Xclaim, ZeroCorss only takes 4759 ms to handle one exchange.

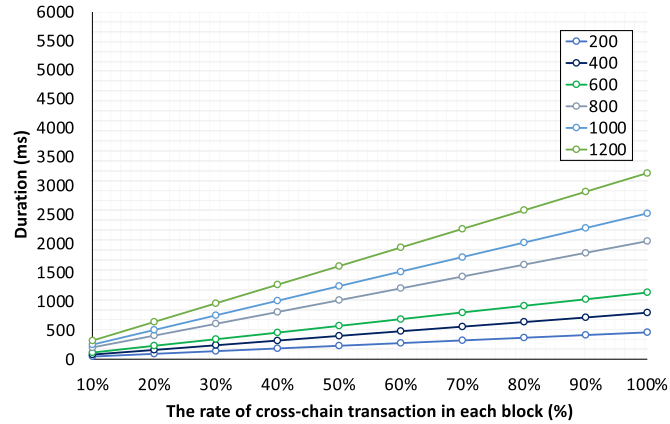


Fig. 7. The time difference caused by the proof of zero-knowledge protocol with the number of transaction ranging from 200 to 1200.

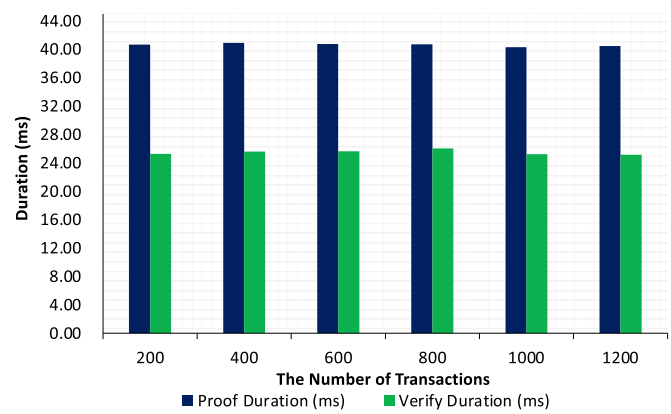


Fig. 8. The impact of the proportion of cross-chain transaction on the time required for miners to verify the proof.

Verification and proof time of payers with increasing transactions. For measuring the overhead caused by the increment of the number of transactions in a block, we evaluate the change in verification time and proof generation time as the number of transactions increases from 200 to 1200. As shown in Fig. 8, the verification time and the proof time have barely changed with the increment of the number of transactions. Thus, its overhead of it can be negligible.

Verification time of each miner with increasing transactions. Further, we measure the verification time of each miner with the proportion of transactions relating to the cross-chain exchange payment in a block increasing from 10% to 100% when the number of transactions is 200, 400, 600, 800, 1000 and 1200. As shown in Fig. 7, the execution time increases linearly with the increment of the proportion, and the execution time for different numbers of transactions increases gradually. Specifically, even if the proportion of transactions is 100% and the number of transactions is 1200, the verification time is acceptable (only 3.2 seconds).

Table 2
Computation cost.

	Payer A	Payer B	Miners
Contract setup	1.77 ± 0.02 ms	1.78 ± 0.01 ms	4.993 ± 0.04 ms
Swap Monero	40.04 ± 0.85 ms	8.33 ± 0.09 ms	2324.99 ± 86.97 ms
Swap Zerocash	8.34 ± 0.05 ms	40.05 ± 0.84 ms	2324.99 ± 75.50 ms
Final Determination	0	0.01 ± 0 ms	3.34 ± 0.04 ms

Total	Xclaim [47]	ZeroCross
	358800 ms	4758.63 ms

Table 3
The communication cost.

Phases	Communication Cost
Contract setup	$6\lfloor G\rfloor + 2\lfloor Z\rfloor + 2\lfloor Proof\rfloor$
Swap Monero	$\lfloor Proof\rfloor + \lfloor Sign\rfloor + \lfloor G\rfloor$
Swap Zerocash	$\lfloor Proof\rfloor + \lfloor Sign\rfloor + \lfloor G\rfloor + \lfloor Z\rfloor$
Final Determination	$\lfloor Z\rfloor$

Table 4
Main parameters of popular cryptocurrencies.

	Ethereum	ZeroCash	Monero	Bitcoin
Generation rate (s/per)	∼ 15	∼ 150	∼ 120	∼ 600
Confirmation (block)	∼ 12	∼ 12	∼ 32	∼ 6
Confirmation time (s)	∼ 180	∼ 1800	∼ 3840	∼ 3600

Compared to the computation cost between payers and miners, we can observe that the computation cost of miners is higher than the computation cost of payers but still acceptable during our protocol.

7.2. Communication cost

As shown in Table 3, we illustrate the theoretical communication cost concerning each phase of our protocol. We only analyze the communication cost among a pair of payers and one miner but do not consider the communication cost caused among miners. Specifically, we denote a series of notations as follows: 1) $\lfloor G\rfloor$ is denoted as the size of an element in the cyclic group where the outputs of the commitment scheme and the ElGamal encryption scheme belong to, 2) $\lfloor Z\rfloor$ is defined as the size of an element in group Z, 3) $\lfloor Sign\rfloor$ is the size of the output of the signature algorithm, and 4) $\lfloor Proof\rfloor$ is regarded as the size of a proof generated by the CP-SNARK framework. As we can see in Table 3, the main communication cost happens in the top three phases. In the contract setup phases, the communication cost is incurred because both payers need to submit a ciphertext of a partial key of a shared account in Monero and a proof of the correctness of ciphertext to ZeroCross. Payers need to generate signatures, commitments, and proof of membership in the swap phases. Finally, the communication cost in the final determination phase only depends on the size of a partial key decrypted to a payer which belongs to the group Z.

7.3. Time performance

In terms of the experiment results and the current state of various blockchains as shown in Table 4, we analyze the time performance of our proposed solution. During our solution, time performance can be computed following the format:

$$\text{Latency} = \text{computation cost} + \text{communication cost} + \text{confirmation time}.$$

The communication costs depend primarily on the network conditions, focusing on confirmation time and computation cost. During our solution, payers need to monitor whether the transactions in the underlying blockchains are confirmed or not and generate the proof of membership for convincing the miners in ZeroCross that the transaction is correct and sealed in a confirmation block. Thus, the confirmation time of the transaction has a significant influence on one-time performance. Take Ethereum, ZeroCash, Bitcoin, and Monero as examples; their confirmation time ranges from 180 s to 3840 s. Comparing the confirmation time shown in Table 4 with the computation cost shown in the Table 2, the computation cost is far lower than the confirmation time. Thus, the time performance mainly depends on the confirmation time required by the underlying blockchains. That is, the overhead of our solution is negligible for the existing systems.

8. Related work

Our work is closely related to previous mechanisms [27,46,42,36] which study cross-chain cryptocurrency exchange between separate blockchains, such as Ethereum and Bitcoin. The mechanisms are widely classified into three categories from literature, including sidechain/relay solutions [26,11,46,17], notary schemes [21,36], and HTLC [15,28,34], according to the technologies they leverage. Besides, we are also aware of some prior efforts specific for Monero [18,15,23]. However, they do not address privacy-preserving issues we are concerned about or are only suitable for limited scenarios in this paper. Now, we are ready to introduce previous mechanisms on a category-by-category basis.

Sidechain/Relay. A sidechain is usually affiliated to an existing blockchain (called *main chain*) while it is technically separated from the main chain. Thus, the consensus mechanism and the cryptocurrencies in the sidechain are independent of the main chain [25]. Such a design realizes a two-way peg [13], in which cryptocurrencies across blockchains can be transferred not only in the main chain but also on the sidechain. Specifically, Cosmos [26] is one of the most popular sidechain solutions for cross-chain cryptocurrency exchange. It introduces two primary roles: (a) a hub responsible for coping with cross-chain interactions; (b) multiple zones that are essentially parallel blockchains and communicate via a consensus protocol named IBC for exchanging cross-zone cryptocurrencies.

BTC-Relay [11] builds a small Bitcoin-like blockchain based on block headers and enables the Ethereum-based network to interoperate with the Bitcoin network. Manta [12], Xclaim [47], and Haveno [20] require users to lock some cryptocurrencies in the underlying blockchains as deposits to issue cryptocurrencies in the sidechain. Tari [41] depends on the Tor network to realize the privacy of cross-chain exchange and also inherit the limitations of the Tor network. Compared with previously mentioned solutions [26,11], Teechain [27], and Tesseract [4] attempt to improve the efficiency of cross-chain cryptocurrency exchange. Concretely, Teechain and Tesseract employ trusted hardware empowered components to enforce secure exchanging of off-chain assets between asynchronous blockchains while enabling real-time asset swapping. Despite the efforts (*e.g.*, force-freeze replication) to defend against the failures of trusted hardware, requiring that each blockchain

user is empowered with a trusted hardware-based component might be a strong assumption. Thus, it might not be acceptable for the asset exchanging scenario and result in additional costs to the users. To meet the privacy concern, [17] proposed a cross-chain protocol named "Zendoo" based on zk-SNARK [32] to protect the private information of the exchange. However, it requires that the sidechain and the main chain monitor each other constantly, and the sidechain is based on a unique construction that may be difficult to instantiate in reality.

Essentially, the mentioned-above schemes resort to a sidechain to verify whether the transactions issued by the cross-chain operations are correct. Such approaches make the private information about the exchange accessible to every blockchain user, and the identification information regarding the exchange thus can be leaked. The information about the exchange is exposed to the public conflicts with the privacy requirements of Monero. Different from the previous schemes, this paper provides a publicly verifiable tool [3] for us to construct a proof of membership protocol for ensuring the correctness of exchanging transactions while fully respecting the privacy of the transactions. Further, our solution stimulates payers to update the condition by themselves instead of requiring that miners constantly monitor the sidechain and the main chains, which can relieve the miners' burden caused by the exchange. Our work also does not require each payer to be equipped with the trusted hardware, which reduces the limiting condition of using our solution.

Notary Schemes. Notary schemes [21,36] usually rely on trusted third parties or communities to enforce the correctness of the exchanged transactions between the underlying blockchains. Specifically, Corda [21] requires two exchanging blockchains to negotiate a group of trusted third parties as notaries who are responsible for synchronizing the exchanging data on the two blockchains. Ripple [36] instead introduces a cryptocurrency system called XMR as an intermediary to exchange the cryptocurrencies of the underlying exchanging blockchains.

Despite the feasibility of the notary schemes, it might not be efficient since they do not consider the privacy design of Monero, and privacy information about the exchange is public to the intermediaries in the notary schemes. Different from those previous works, our work builds a distributed and privacy-preserving system called ZeroCross. Based on the zero-knowledge proof protocol [6,7,32], ZeroCross enables the correctness verification of cross-chain operations, and meanwhile, preserves the exchanging privacy.

Hashed TimeLock Contract. HTLC-based solutions [15,28] primarily rely on hash locks and time locks to deploy the exchanging of cross-chain assets. The critical insight of HTLC enabled solutions is to build up several payment channels between payers based on HTLC. HTLC can be traced back to the design in the Lightning network [34]. These payment channels form a chain so that the first payer in the chain can exchange the cross-chain assets with the last payer [13]. Deshpande [15], and Malavolta Giulio [28] propose privacy-preserving HTLC-based protocols, with the same goal to protect transaction privacy including the transferred values privacy and identity privacy. However, HTLC-based protocols are limited to specific blockchains that support hash lock, while most privacy-preserving blockchains like Monero have no mechanism [40].

Cross-chain Exchange Mechanisms for Monero. Recent cross-chain cryptocurrency exchange mechanisms [18,23,1] suitable for Monero are still not really satisfactory. Specifically, the solutions proposed by Gugger [18] and Hoenisch and del Pino [23] depend on the signature scripts in bitcoin, aiming to enable the swapping of Bitcoin and Monero; such technical dependence makes it hard to support other cryptocurrencies which lack the support of signature scripts. Similarly, [40] might only enable exchanging Monero with cryptocurrencies that support a smart-contract-like mechanism or a timelock mechanism for asset locking, so it might fail to support cryptocurrencies like ZeroCash and ZeroCoin. Teechain [27] requires that each user is equipped with Trusted Execution Environment, which makes it unsuitable for the users without TEE to exchange cryptocurrencies. Another privacy-preserving cross-chain protocol proposed by Baum et al. [1] breaks the linkability of the transactions in the underlying blockchains during the cross-chain exchange and protects the order privacy from intermediaries, which is close to our design goals. [43] proposes a universal atomic swap scheme considering protecting the private information of cross-chain exchanges via transferring the process of cross-chain exchange to offline. However, it is unsuitable for sidechain-based solutions because those solutions have to record the exchange information on the sidechain so that the private information can not be protected. Unlike our work, this protocol only guarantees order privacy when multiple cross-chain exchanges are initiated simultaneously.

Compared with the aforementioned work, ZeroCross can enable various cryptocurrencies to satisfy more scenario requirements than existing mechanisms.

9. Conclusion

In this paper, we propose ZeroCross, a privacy-preserving cross-chain cryptocurrency exchange solution for Monero. By leveraging our protocol, payers can exchange cryptocurrencies with Monero without revealing any details of the transaction in the underlying blockchain. We also utilize zero-knowledge proofs of knowledge and accumulators for defending the remote side-channel attack in the cross-chain cryptocurrency exchange and proving this for ZeroCross in the UC framework. Finally, we evaluate the functionality of ZeroCross, describing our parameter and design choices.

CRediT authorship contribution statement

Yuxian Li: Conceptualization, Methodology, Software, Investigation, Formal Analysis, Writing – Original Draft, Drafting the manuscript, Approval of the version of the manuscript to be published (the names of all authors must be listed). **Jian Weng**: Conceptualization, Funding Acquisition, Resources, Supervision, Writing – Review & Editing, Drafting the manuscript, Revising the manuscript critically for important intellectual content, Approval of the version of the manuscript to be published (the names of all authors must be listed). **Ming Li:** Data Curation, Writing – Original Draft, Drafting the manuscript, Approval of the version of the manuscript to be published (the names of all authors must be listed). **Wei Wu:** Visualization, Investigation, Revising the manuscript critically for important intellectual content, Approval of the version of the manuscript to be published (the names of all authors must be listed). **Jiasi Weng:** Resources, Supervision, Approval of the version of the manuscript to be published (the names of all authors must be listed). **Jia-Nan Liu:** Software, Validation, Revising the manuscript critically for important intellectual content, Approval of the version of the manuscript to be published (the names of all authors must be listed). **Shun Hu:** Visualization, Writing – Review & Editing, Revising the manuscript critically for important intellectual content, Approval of the version of the manuscript to be published (the names of all authors must be listed).

Declaration of competing interest

The author(s) declare(s) that there are no conflicts of interest regarding the publication of this article.

Acknowledgment

This work is supported by the National Key R&D Plan of China (Grant Nos. 2020YFB1005600), National Natural Science Foundation of China (Grant Nos. 61825203, 62102165, 62032025, and U2001205), Major Program of Guangdong Basic and Applied Research Project (Grant No. 2019B030302008, 2020A1515111175), and Guangdong Provincial Science and Technology Project (Grant Nos. 2019B010137002 and 2017B010111005) and China Scholarship Council (Grant No. 202106780013).

References

[1] C. Baum, B. David, T.K. Frederiksen, P2dex: privacy-preserving decentralized cryptocurrency exchange, in: International Conference on Applied Cryptography and Network Security (ACNS), Springer, 2021, pp. 163–194.
[2] R. Belchior, A. Vasconcelos, S. Guerreiro, M. Correia, A survey on blockchain interoperability: past, present, and future trends, ACM Comput. Surv. 54 (8) (2021) 1–41.
[3] D. Benarroch, M. Campanelli, D. Fiore, D. Kolonelos, Zero-Knowledge Proofs for Set Membership: Efficient, Succinct, Modular IACR Cryptol ePrint Arch, 2019/1255 2019.
[4] I. Bentov, Y. Ji, F. Zhang, L. Breidenbach, P. Daian, A. Juels, Tesseract: real-time cryptocurrency exchange using trusted hardware, in: Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security (CCS), ACM, 2019, pp. 1521–1538.
[5] D. Boneh, B. Bünz, B. Fisch, Batching techniques for accumulators with applications to iops and stateless blockchains, in: Annual International Cryptology Conference, Springer, 2019, pp. 561–586.
[6] S. Bowe, A. Chiesa, M. Green, I. Miers, P. Mishra, H. Wu, Zexe: enabling decentralized private computation, in: 2020 IEEE Symposium on Security and Privacy (S&P), IEEE, 2020, pp. 947–964.
[7] B. Bünz, J. Bootle, D. Boneh, A. Poelstra, P. Wuille, G. Maxwell, Bulletproofs: short proofs for confidential transactions and more, in: 2018 IEEE Symposium on Security and Privacy (S&P), IEEE, 2018, pp. 315–334.
[8] J. Camenisch, A. Lysyanskaya, Dynamic accumulators and application to efficient revocation of anonymous credentials, in: Annual International Cryptology Conference, Springer, 2002, pp. 61–76.
[9] R. Canetti, Universally composable security: a new paradigm for cryptographic protocols, in: Proceedings 42nd IEEE Symposium on Foundations of Computer Science, IEEE, 2001, pp. 136–145.
[10] B. Chor, S. Goldwasser, S. Micali, B. Awerbuch, Verifiable secret sharing and achieving simultaneity in the presence of faults, in: 26th Annual Symposium on Foundations of Computer Science (SFCS), IEEE, 1985, pp. 383–395.
[11] J. Chow, Btc Relay, btc-relay, 2016.
[12] S. Chu, Y. Xia, Z. Zhang, Manta: A Plug and Play Private Defi Stack, Cryptology ePrint Archive, 2021.
[13] L. Deng, H. Chen, J. Zeng, L.J. Zhang, Research on cross-chain technology based on sidechain and hash-locking, in: 2018 International Conference on Edge Computing (ICEC), Springer, 2018, pp. 144–151.
[14] D. Derler, C. Hanser, D. Slamanig, Revisiting cryptographic accumulators, additional properties and relations to other primitives, in: Cryptographers' Track at the RSA Conference (CT-RSA), Springer, 2015, pp. 127–144.
[15] A. Deshpande, M. Herlihy, Privacy-preserving cross-chain atomic swaps, in: International Conference on Financial Cryptography and Data Security (FC), Springer, 2020, pp. 540–549.
[16] T. ElGamal, A public key cryptosystem and a signature scheme based on discrete logarithms, IEEE Trans. Inf. Theory 31 (4) (1985) 469–472.
[17] A. Garoffolo, D. Kaidalov, R. Oliynykov, Zendoo: a zk-snark verifiable cross-chain transfer protocol enabling decoupled and decentralized sidechains, in: 2020 IEEE 40th International Conference on Distributed Computing Systems (ICDCS), IEEE, 2022, pp. 1257–1262.
[18] J. Gugger, Bitcoin–monero cross-chain atomic swap, Tech. Rep., 2020.
[19] K. Gurkan, Cpsnarks set, https://github.com/kobigurk/cpsnarks-set/, 2022. (Accessed 23 May 2022).
[20] Haveno, https://haveno.exchange/, 2022. (Accessed 23 May 2022).
[21] M. Hearn, Corda: a distributed ledger, Corda Technical White Paper, 2016.
[22] R. Henry, A. Herzberg, A. Kate, Blockchain access privacy: challenges and directions, IEEE Secur. Priv. 16 (4) (2018) 38–45.
[23] P. Hoenisch, L.S. del Pino, Atomic swaps between bitcoin and monero, IACR Cryptol ePrint Arch, 2020/1126, 2020.
[24] M. Jakobsson, A. Juels, Mix and match: secure function evaluation via ciphertexts, in: International Conference on the Theory and Application of Cryptology and Information Security, Springer, 2000, pp. 162–177.
[25] N. Kannengießer, M. Pfister, M. Greulich, S. Lins, A. Sunyaev, Bridges between islands: cross-chain technology for distributed ledger technology, in: Proceedings of the 53rd Hawaii International Conference on System Sciences (HICSS), 2020, pp. 5298–5307.
[26] J. Kwon, E. Buchman, Cosmos Whitepaper, 2019.
[27] J. Lind, O. Naor, I. Eyal, F. Kelbert, E.G. Sirer, P. Pietzuch, Teechain: a secure payment network with asynchronous blockchain access, in: Proceedings of the 27th ACM Symposium on Operating Systems Principles (SOSP), ACM, 2019, pp. 63–79.
[28] G. Malavolta, P. Moreno-Sanchez, C. Schneidewind, A. Kate, M. Maffei, Anonymous Multi-Hop Locks for Blockchain Scalability and Interoperability, IACR Cryptol ePrint Arch, 2018/472, 2018.
[29] Monero coin trading quotes, https://cn.investing.com/crypto/monero/markets, 2022. (Accessed 23 May 2022).
[30] S. Noether, Ring Signature Confidential Transactions for Monero, IACR Cryptol ePrint Arch, 2015/1098, 2015.
[31] A. Ozdemir, R. Wahby, B. Whitehat, D. Boneh, Scaling verifiable computation using efficient set accumulators, in: 29th USENIX Security Symposium (USENIX Security), USENIX Association, 2020, pp. 2075–2092.
[32] B. Parno, J. Howell, C. Gentry, M. Raykova, Pinocchio: nearly practical verifiable computation, in: 2013 IEEE Symposium on Security and Privacy (S&P), IEEE, 2013, pp. 238–252.
[33] T.P. Pedersen, Non-interactive and information-theoretic secure verifiable secret sharing, in: Annual International Cryptology Conference, Springer, 1991, pp. 129–140.
[34] J. Poon, T. Dryja, The Bitcoin Lightning Network: Scalable Off-Chain Instant Payments, 2016.
[35] C.P. Schnorr, Efficient identification and signatures for smart cards, in: Conference on the Theory and Application of Cryptology, Springer, 1989, pp. 239–252.
[36] D. Schwartz, N. Youngs, A. Britto, et al., The ripple protocol consensus algorithm, Ripple Labs. Inc. White Pap. 5 (8) (2014).
[37] A. Shamir, How to share a secret, Commun. ACM 22 (11) (1979) 612–613.
[38] A. Singh, K. Click, R.M. Parizi, Q. Zhang, A. Dehghantanha, K.K.R. Choo, Sidechain technologies in blockchain networks: an examination and state-of-the-art review, J. Netw. Comput. Appl. 149 (2020) 102471.
[39] S.F. Sun, M.H. Au, J.K. Liu, T.H. Yuen, Ringct 2.0: a compact accumulator-based (linkable ring signature) protocol for blockchain cryptocurrency monero, in: European Symposium on Research in Computer Security, Springer, 2017, pp. 456–474.
[40] E. Tairi, P. Moreno-S'anchez, M. Maffei, A 2 l: anonymous atomic locks for scalability in payment channel hubs, in: 2021 IEEE Symposium on Security and Privacy (S&P), IEEE, 2021, pp. 1834–1851.
[41] Tari, https://www.tari.com/, 2022. (Accessed 23 May 2022).
[42] S. Thomas, E. Schwartz, A protocol for interledger payments, Tech. Rep., 2015.
[43] S.A. Thyagarajan, G. Malavolta, P. Moreno-Sánchez, Universal Atomic Swaps: Secure Exchange of Coins Across All Blockchains, IACR Cryptol ePrint Arch, 2021/1612, 2021.
[44] F. Tramèr, D. Boneh, K. Paterson, Remote side-channel attacks on anonymous transactions, in: 29th USENIX Security Symposium (USENIX Security), USENIX Association, 2020, pp. 2739–2756.
[45] J. Weng, J. Weng, J. Zhang, M. Li, Y. Zhang, W. Luo, Deepchain: auditable and privacy-preserving deep learning with blockchain-based incentive, IEEE Trans. Dependable Secure Comput. 18 (21076084) (2019) 2438–2455.
[46] G. Wood, Polkadot: vision for a heterogeneous multi-chain framework, White Pap. (2016).
[47] A. Zamyatin, D. Harz, J. Lind, P. Panayiotou, A. Gervais, W. Knottenbelt, Xclaim: trustless, interoperable, cryptocurrency-backed assets, in: 2019 IEEE Symposium on Security and Privacy (S&P), IEEE, 2019, pp. 193–210.

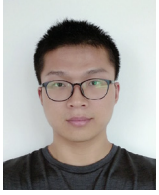
Yuxian Li obtained the B.S degree in Computer Science and Technology from Jinan University in June 2018. Currently, she is a Ph.D. student with School of Information Science and Technology in Jinan University. Her research interests include applied cryptography and blockchain.

Jian Weng is a professor and the Executive Dean with College of Information Science and Technology in Jinan University. He received B.S. degree and M.S. degree from South China University of Technology in 2001 and 2004 respectively, and Ph.D. degree at Shanghai Jiao Tong University in 2008. His research areas include public key cryptography, cloud security, blockchain, etc. He has published 80 papers in international conferences and journals such as CRYPTO, EUROCRYPT, ASIACRYPT, TCC, PKC, CT-RSA, IEEE TPAMI, IEEE TDSC, etc.

Ming Li received his B.S. in electronic information engineering from University of South China in 2009, and M.S. in information processing from Northwestern Polytechnical University in 2012. From 2016, he started his Ph.D. at Jinan University. His research interests include blockchain, crowdsourcing and its privacy and security.

Wei Wu received his B.S. degree from the college of Mathematics and Informatics, South China Agricultural University, in 2018. And he is currently pursuing the M.S. degree with the College of Cyber Security, Jinan University. His research interests include cryptography and blockchain.

Jiasi Weng obtained the B.S degree in Software engineering from South China Agriculture University in June 2016. Currently, she is a Ph.D. student with School of Information Science and Technology in Jinan University. Her research interests include applied cryptography and blockchain.



Jia-Nan Liu received the Ph.D. degree and M.S. degree from Jinan University in 2020 and 2016, respectively, and the B.S. degree from Zhengzhou University in 2013. He is currently a postdoctoral researcher in Jinan University. His research interesting includes cryptography, smart grid security and cloud computing security.

Hu Shun received the B.Eng. degree in electrical information engineer from University of Electronic Science and Technology of China in 2011, and the M.Sc. degree in computer application technology from Jinan University in 2015. Now, he is a Ph.D student at Jinan University. His current research interests include secure muti-party computation and IOT security.